AI-BASED ELECTRICITY DEMAND INTELLIGENCE SYSTEM FOR SRI LANKA

INDIVIDUAL THESIS

Individual component: Behaviour Intelligence

Group ID: R26-IT-010

Thanujan T

(IT22545176)

Report submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) in Information Technology Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Thanujan T

Student ID: IT22545176

Signature: _____ Date: ____8/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Nathali Silva		
Co-supervisor: [Name]		

Abstract

Electricity planners in Sri Lanka receive demand figures and forecasts, but little help in understanding what kind of day each day was, which days behaved abnormally, and why. This thesis presents the Behaviour Intelligence component of the AI-Based Electricity Demand Intelligence System for Sri Lanka (Project R26-IT-010). The component converts a 15-minute load series (189,793 readings, January 2020 to May 2025) into 1,977 daily 96-point profiles and gives every day a cluster, an anomaly score, a peak flag and one of four behaviour labels (Normal Weekday, Peak, Holiday / Low and Abnormal Demand Day) with a risk level and a written reason.

Following Design Science Research, four clustering algorithms were ranked by their mean rank on the Silhouette, Davies–Bouldin and Calinski–Harabasz indices, and four anomaly detectors by the bootstrap stability of their flagged days. K-Means with k = 2 was selected (Silhouette 0.4729, Davies–Bouldin 0.8373); its clusters match weekdays and weekends on 99.14% of days. Isolation Forest flagged 60 days with near-ceiling stability (mean Jaccard 0.7769 against 0.80), and Local Outlier Factor and DBSCAN noise flagged exactly the same days. The labels gave 1,279 normal, 281 peak, 396 holiday/low and 21 abnormal days, and a frequency-based module predicts the label of a future date with 78.85% accuracy against a 62.36% baseline.

The component is delivered as a FastAPI service with 12 endpoints and a React dashboard with model-comparison pages that also integrates the team's other applications; 24 of 24 API checks passed and the saved results were reproduced exactly. The evaluation also reports its limits: the dataset is synthetic with undocumented provenance, no ground-truth events exist for precision and recall, outlier cleaning removes demand spikes, and the peak rule labels 69 low-demand weekend days as peak days.

Keywords: Electricity demand behaviour, daily load profiles, clustering, anomaly detection, Sri Lanka

ACKNOWLEDGEMENTS

I sincerely thank my research supervisor, Dr. Nathali Silva, and my co-supervisor, [co-supervisor name], for their guidance, expert advice and constructive feedback throughout this study. Their direction helped me turn a broad idea about demand behaviour into a component that could be built, measured and explained.

I am grateful to the lecturers and staff of the Department of Information Technology and the Faculty of Computing at SLIIT for the knowledge, resources and support that made this research possible.

I thank my team members of Project R26-IT-010, Madushani R.P. (IT22273062), Janahan R. (IT22069740) and Dewmini D.M.D.P. (IT22630216), for their cooperation and for the shared work that allowed my component to be integrated with theirs.

Finally, I thank my family and friends for their encouragement, patience and support. [State any funding or sponsorship; otherwise confirm that the research was self-funded.]

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION 1
Abstract 1
ACKNOWLEDGEMENTS 2
TABLE OF CONTENTS 3
LIST OF FIGURES 4
LIST OF TABLES 4
LIST OF ABBREVIATIONS 5
LIST OF APPENDICES 6
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 3
1.4 Research Gap 4
1.5 Aim and Objectives 4
1.6 Research Questions or Hypotheses 5
1.7 Scope, Limitations and Assumptions 6
1.8 Individual Contribution 6
1.9 Thesis Structure 7
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review Method and Scope 9
2.2 Domain and Theoretical Foundations 9
2.2.1 Daily load profiles and demand behaviour 9
2.2.2 Clustering methods 10
2.2.3 Internal cluster validity 10
2.2.4 Unsupervised anomaly detection 10
2.2.5 Selecting unsupervised models without ground truth 11
2.3 Related Research 11
2.3.1 Demand forecasting in Sri Lanka 11
2.3.2 Clustering of load patterns 12
2.3.3 Anomaly detection in electricity consumption 12
2.4 Existing Systems, Products and Techniques 13
2.5 Gap Synthesis and Conceptual Framework 14
3 RESEARCH METHODOLOGY 16
3.1 Research Design 16
3.2 Data, Participants or Study Materials 17
3.2.1 Input variables 17
3.2.2 Replacement of the earlier dataset 18

3.2.3 Structure of the data 18
3.2.4 Preprocessing 18
3.3 Experimental or Study Procedure 19
3.3.1 Daily profiles and scaling 20
3.3.2 Clustering and choice of k 20
3.3.3 DBSCAN with a data-driven radius 21
3.3.4 Anomaly detection 21
3.3.5 Peak-day detection 21
3.3.6 Behaviour labelling 22
3.3.7 Model-comparison procedures 23
3.3.8 Future-behaviour prediction 23
3.4 Measures and Evaluation Metrics 24
3.5 Data Analysis 24
3.6 Ethics, Privacy, Security and Responsible AI 25
3.7 Reproducibility and Research Artefacts 25
4 PRODUCT AND SOLUTION DESIGN 27
4.1 Requirements 27
4.1.1 Stakeholders 27
4.1.2 Functional requirements 27
4.1.3 Non-functional requirements 28
4.2 Component Architecture within the Complete Solution 28
4.3 Design Alternatives and Decisions 29
4.4 Detailed Design 30
4.4.1 Input representation 30
4.4.2 Analysis modules 31
4.4.3 Labelling and explanation 31
4.4.4 Model-comparison services 32
4.4.5 Service interface 32
4.4.6 User interface 33
4.5 Standards and Professional Practice 33
5 IMPLEMENTATION, INTEGRATION AND TESTING 34
5.1 Development Environment and Technology Stack 34
5.2 Component Implementation 34
5.3 Integration with the Other Components 41
5.4 Verification and Software Testing 42
5.5 Deployment and Operation 43
6 RESULTS AND VALIDATION 45
6.1 Research Results 45
6.1.1 Experimental setting 45

6.1.2 Choice of the number of clusters 45
6.1.3 Comparison of clustering algorithms 46
6.1.4 What the clusters represent 47
6.1.5 Behaviour of DBSCAN 48
6.1.6 Comparison of anomaly detectors 49
6.1.7 Characteristics of the anomalous days 50
6.1.8 Peak-demand detection 51
6.1.9 Behaviour labels 52
6.1.10 Future-behaviour prediction 54
6.2 Product and Technical Results 55
6.3 User or Stakeholder Validation 56
6.4 Integration Results 56
6.5 Summary of Findings 57
7 DISCUSSION 58
7.1 Interpretation of Findings 58
7.2 Comparison with Literature Baselines and Existing Solutions 59
7.3 Research Contribution 60
7.4 Product Contribution 60
7.5 Limitations and Threats to Validity 61
7.6 Implications 62
8 COMMERCIALISATION AND SUSTAINABILITY 63
8.1 Value Proposition and Target Users 63
8.2 Market and Competitive Position 63
8.3 Business or Sustainability Model 64
8.4 Scalability, Intellectual Property and Compliance 64
8.5 Contribution to the Sustainable Development Goals 65
9 CONCLUSION AND FUTURE DIRECTIONS 66
9.1 Conclusion 66
9.2 Achievement of Objectives 67
9.3 Individual Research and Product Contributions 67
9.4 Future Directions 67
REFERENCES 69
GLOSSARY 72
APPENDICES 73
Appendix A Extended Results and Evidence Record 73
A.1 Proposal plan compared with the implementation 73
A.2 Evidence sources in the project repository 73
A.3 Example service responses 74
Appendix B Academic Integrity and AI Use Disclosure 75

LIST OF FIGURES

Figure 1.1: High-level architecture of the AI-Based Electricity Demand Intelligence System (original in colour)..... 2
Figure 3.1: Design Science Research activities applied in this study (original in colour)..... 16
Figure 3.2: Analysis pipeline of the behaviour intelligence component (original in colour)..... 20
Figure 3.3: Ordered rules that assign the behaviour label (original in colour)..... 22
Figure 4.1: Internal architecture of the behaviour intelligence component (original in colour)..... 29
Figure 5.1: Dashboard: sidebar, summary cards, behaviour-label distribution and risk-level distribution (original in colour)..... 35
Figure 5.2: Dashboard: the 15 most anomalous days and the anomaly-score trend for the last 180 days (original in colour)..... 36
Figure 5.3: Dashboard: day explorer with the detail panel of a selected day (original in colour)..... 36
Figure 5.4: Dashboard: 96-point load profile of the selected day (original in colour)..... 36
Figure 5.5: Dashboard: recent abnormal days with reasons and links to the LIME explanation in the XAI component (original in colour)..... 37
Figure 5.6: Dashboard: future-behaviour prediction for a selected date (original in colour)..... 37
Figure 5.7: Model Comparison page: model cards, selection explanation and side-by-side metrics (original in colour)..... 38
Figure 5.8: Model Comparison page: Silhouette and Davies–Bouldin charts, ranking and selected model (original in colour)..... 39
Figure 5.9: Anomaly Comparison page: detector cards, methodology note, selection explanation and side-by-side metrics (original in colour)..... 40
Figure 5.10: Anomaly Comparison page: stability charts, ranking and Spearman rank-agreement diagnostic (original in colour)..... 41
Figure 6.1: Silhouette, Davies–Bouldin and inertia of K-Means for k = 2 to 12 (original in colour)..... 46
Figure 6.2: Validity indices of the clustering algorithms at k = 2 (original in colour)..... 47
Figure 6.3: Composition of the K-Means clusters by day of week (original in colour)..... 48
Figure 6.4: Stability of the anomaly detectors and overlap of their flagged days (original in colour)..... 49
Figure 6.5: Behaviour label distribution and label mix by day of week (original in colour)..... 53

Figure 6.6: Behaviour labels by calendar month, January 2020 to May 2025 (original in colour) 53
Figure 6.7: Per-label scores and confusion matrix of the prediction back-test (original in colour) 54
Figure 6.8: Run time of the pipeline stages in two runs (original in colour) 56

LIST OF TABLES

Table 1.1: Components of the complete solution and the component reported in this thesis 2
Table 1.2: Research questions and associated hypotheses 5
Table 1.3: Individual contributions of the author 7
Table 2.1: Summary of selected related studies and their relationship to this work 13
Table 2.2: Comparison of technique families for day-level behaviour analysis 13
Table 2.3: Mapping of research gaps to objectives and design elements 14
Table 3.1: Summary of the dataset used by the component 17
Table 3.2: Structural diagnostics of the earlier and current datasets 18
Table 3.3: Preprocessing steps and their effect on the dataset 19
Table 3.4: Behaviour labelling rules 22
Table 3.5: Evaluation measures used in this study 24
Table 3.6: Reproducibility checklist and status 26
Table 4.1: Functional requirements and implementation status 27
Table 4.2: Non-functional requirements and how they are addressed 28
Table 4.3: Key design decisions and justification 30
Table 4.4: Algorithms in the component and their configuration 31
Table 4.5: REST endpoints of the behaviour intelligence service 32
Table 5.1: Technology stack and justification 34
Table 5.2: Components started by the team launcher 41
Table 5.3: Verification checks and results 42
Table 6.1: K-Means validity indices, inertia and smallest cluster for k = 2 to 12 45
Table 6.2: Comparison of clustering algorithms on the daily profiles 46
Table 6.3: Profile of the two K-Means clusters 47
Table 6.4: Comparison of anomaly detectors 49
Table 6.5: The ten highest-scoring anomalous days 51
Table 6.6: Conditions of the peak-day rule and the days that satisfy them 51
Table 6.7: Effect of the absolute z-score in the peak rule on the label counts 52
Table 6.8: Behaviour label distribution and characteristics 52
Table 6.9: Back-test of the future-behaviour prediction on a temporal hold-out 54
Table 6.10: Measured API response times 55
Table 6.11: Summary of findings and strength of evidence 57
Table 7.1: Limitations and threats to validity 61
Table 8.1: Planned project budget (LKR) 64
Table 9.1: Achievement of the objectives 67
Table A.1: Proposal plan compared with the implemented component 73
Table A.2: Project artefacts that hold the reported evidence 73

Table B.1: Disclosure of generative AI use 75

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BI	Behaviour Intelligence
CEB	Ceylon Electricity Board
CH	Calinski–Harabasz score
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DB / DBI	Davies–Bouldin Index
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DSR	Design Science Research
FR	Functional Requirement
GMM	Gaussian Mixture Model
IF	Isolation Forest
IQR	Inter-Quartile Range
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LECO	Lanka Electricity Company
LIME	Local Interpretable Model-agnostic Explanations
LKR	Sri Lankan Rupee
LOF	Local Outlier Factor
ML	Machine Learning
NFR	Non-Functional Requirement
OC-SVM	One-Class Support Vector Machine
PCA	Principal Component Analysis
REST	Representational State Transfer
RQ	Research Question
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
STLF	Short-Term Load Forecasting

Abbreviation	Meaning
TLS	Transport Layer Security
UI	User Interface
XAI	Explainable Artificial Intelligence

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Extended Results and Evidence Record	73
Appendix B	Academic Integrity and AI Use Disclosure	75

1 INTRODUCTION

1.1 Overall Project Context

Electricity demand is growing worldwide as industry expands and more end uses are electrified [1]. Because electricity cannot be stored cheaply at grid scale, generation must follow demand closely, and the quality of the information that planners have about demand has a direct operational and financial consequence. In Sri Lanka the Ceylon Electricity Board (CEB) operates a national grid whose generation mix depends heavily on rainfall-fed hydropower, with imported fuel for thermal backup [2], [3]. When dry seasons, fuel shortages or unexpected demand spikes coincide, the grid comes under strain, and the country has experienced load shedding in recent years [4], [5].

Research on electricity demand has concentrated on forecasting: statistical models such as ARIMA and regression were followed by machine-learning and deep-learning models that predict demand more accurately [4], [2]. Forecasts answer the question *how much demand will there be?* They do not answer the questions an operator also asks when reading the numbers: *what kind of day is this, is today behaving normally, and which past days were unusual?* The project proposal for this research argued that this behavioural layer is missing from the tools available to Sri Lankan planners.

The research project AI-Based Electricity Demand Intelligence System for Sri Lanka (Project ID R26-IT-010) responds by combining four services in one decision-support system: behavioural analysis of demand, short-term forecasting, explainable AI, and scenario-based decision support. Each service is built by one member of a team of four undergraduates and exposed through a FastAPI backend to a React.js dashboard used by energy planners, CEB and Lanka Electricity Company (LECO) operators and analysts. Table 1.1 lists the services and Figure 1.1 shows the overall architecture.

Table 1.1: Components of the complete solution and the component reported in this thesis

Component (service)	Main purpose	Responsibility
Behaviour intelligence service	Clustering of daily demand profiles, anomaly and peak-day detection, and interpretable behaviour labels	**Thanujan T (IT22545176), this thesis**
Adaptive forecasting and prediction	Short-term demand forecasting with hybrid deep learning and calibrated intervals	Janahan R. (IT22069740)
Explainable AI (XAI) service	Explanations of demand drivers and anomalies (LIME/SHAP) and reports	Madushani R.P. (IT22273062)
Demand forecasting and scenario service	Forecasting dashboard, scenario simulation and decision support for planners	Dewmini D.M.D.P. (IT22630216)

Source: Project proposal, R26-IT-010, and the team launcher script (start-all.py) [confirm that the mapping of team members to services is correct].

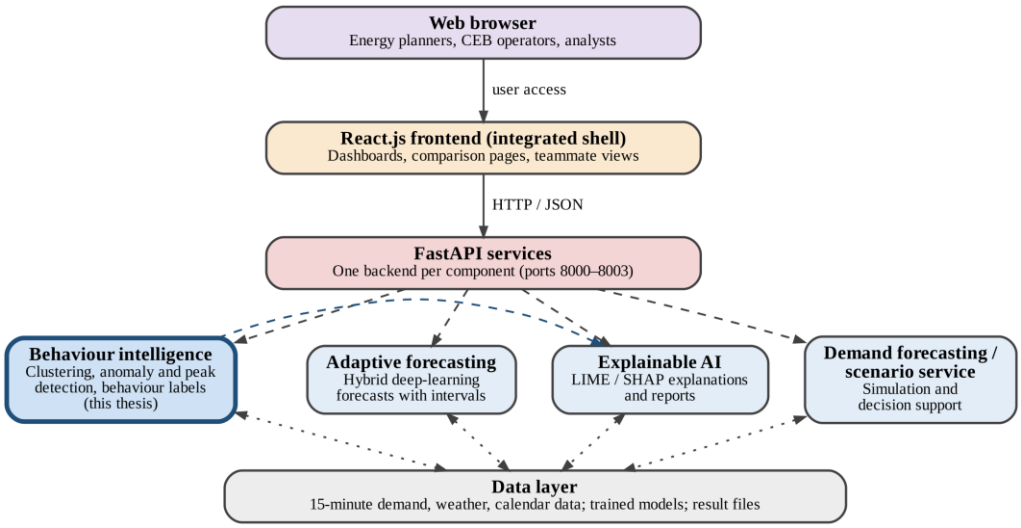


Figure 1.1: High-level architecture of the AI-Based Electricity Demand Intelligence System (original in colour)

Source: Author's drawing based on the project proposal (R26-IT-010) and the implemented services. The behaviour intelligence service, outlined in dark blue, is the subject of this thesis.

1.2 Individual Component Context

This thesis reports the **Behaviour Intelligence** component, the demand-pattern analysis service in the AI layer of the architecture. Its task is to take a 15-minute electricity load series, turn each day into a 96-point load profile, and describe every day in terms a planner can use: which group of similar days it belongs to, whether it is anomalous, whether it is a peak-demand day, and which of four behaviour labels it carries, together with a risk level and a short written reason.

The component is a full-stack service. A Python analysis pipeline (preprocessing, daily profiles, clustering, anomaly detection, peak detection and labelling) writes its results to files; a FastAPI backend with 12 endpoints serves them; and a React dashboard presents them through summary cards, charts, a day explorer, a future-behaviour prediction panel and two model-comparison pages. The same dashboard acts as the integrated shell of the team's system: a sidebar opens the three teammate applications, and each abnormal day links to the explainable-AI service's LIME page for that date.

Two terms are used in a specific sense in this thesis. *Behaviour* refers to the shape and level of a whole day's demand, not to the behaviour of individual consumers. *Label* refers to the output of an explicit, ordered set of rules applied to model outputs (anomaly flag, peak flag and a demand-level threshold); it is not the output of a supervised classifier, because no labelled training data exist.

1.3 Problem Statement

Three weaknesses limit the usefulness of existing demand-analysis work for Sri Lankan planners [4], [5], [3]:

- **Numbers without behaviour.** Forecasting models return demand values but do not tell an operator whether a day is a typical weekday, a low-demand day or an abnormal day, nor why.
- **Fragmented techniques.** Clustering of load profiles is usually applied as a preprocessing step to improve forecasts [6], [7], and anomaly detection is usually applied to individual meters to find faults or fraud [8], [9]. The two are rarely combined into day-level, system-level intelligence.
- **Model choice without ground truth.** Clustering and anomaly detection are unsupervised. Without labelled abnormal days, it is not obvious how to choose between algorithms or how to show that a choice is justified, and published systems seldom explain their choice.

The problem addressed by this thesis is therefore: *how can 15-minute electricity demand data be turned into interpretable day-level behaviour intelligence, by*

comparing unsupervised clustering and anomaly-detection methods in a justified way and combining their outputs into labels that planners can read and act on?

1.4 Research Gap

The project proposal identified the following gaps, which this thesis addresses:

- **Clustering used only as a forecasting aid.** Studies that cluster load profiles mostly do so to improve forecast accuracy [6], [7] or to profile industrial customers or monthly peaks [10], [11], not to describe daily system demand for operators.
- **Anomaly detection at meter level.** Anomaly frameworks target fraud and faults in individual consumers or buildings [8], [12], [9] and do not connect anomalies to a system-level picture of demand behaviour.
- **No interpretable day taxonomy.** No reviewed study or tool assigns each day of a national or utility demand series to a small set of readable behaviour categories with explanations.
- **Little attention to how unsupervised models are selected.** Comparisons often rely on a single index such as the Silhouette score, which can favour degenerate solutions, and anomaly detectors are often compared by how many points they flag, which is set in advance by their contamination parameter.

The novelty claimed in the proposal was a unified pipeline that combines day-level clustering, system-level anomaly detection, peak-day identification and behaviour labelling. The implemented component adds a second contribution: a documented, multi-index procedure for selecting the clustering model and a stability-based procedure for selecting the anomaly detector when no ground truth is available.

1.5 Aim and Objectives

Aim. To design, implement and evaluate a Behaviour Intelligence component that analyses daily electricity demand patterns, detects abnormal and peak-demand days and assigns each day an interpretable behaviour label, and to deliver these outputs as a service within the wider demand-intelligence system.

The aim is pursued through the following objectives, which refine the specific objectives of the proposal:

1. Preprocess the raw 15-minute demand data by handling missing values, removing implausible measurements and standardising the data (proposal objective 2.2.1).
2. Aggregate the cleaned series into 96-point daily demand profiles with daily peak, mean and variability statistics (2.2.2).
3. Apply and compare clustering algorithms, including K-Means and DBSCAN, to group days with similar demand profiles, and select a model with a justified multi-index procedure (2.2.3).
4. Apply and compare anomaly-detection methods, including Isolation Forest and the z-score, to flag abnormal days, and select a detector with a procedure that does not depend on the preset anomaly rate (2.2.4).
5. Generate interpretable behaviour labels (Normal Weekday Demand, Peak Demand Day, Holiday / Low Demand Day and Abnormal Demand Day) with risk levels and written reasons, and expose all outputs through a REST API and an interactive dashboard (2.2.5).
6. Evaluate the component technically and critically, reporting clustering quality, detector stability, the behaviour of the labels, the accuracy of the future-behaviour prediction, software verification, and all limitations that the evaluation reveals (2.2.6).

1.6 Research Questions or Hypotheses

The study is guided by the research questions (RQ) and hypotheses (H) in Table 1.2.

Table 1.2: Research questions and associated hypotheses

ID	Research question	Hypothesis
RQ1	Which clustering algorithm groups the daily demand profiles best when quality is judged on several internal validity indices together?	H1: A partitioning or hierarchical method (K-Means or Ward) ranks better than density-based or mixture models on the combined indices.
RQ2	What structure do the selected clusters represent?	H2: The clusters correspond to recognisable calendar-related demand regimes such as weekdays and weekends.

ID	Research question	Hypothesis
RQ3	Which anomaly detector produces the most stable set of abnormal days, and do independent methods agree on those days?	H3: Isolation Forest gives a stable flagged set, and methods with different principles agree on most of the days it flags.
RQ4	Can rule-based labels built on model outputs give an interpretable and useful description of every day, including likely behaviour on future dates?	H4: Labels and reasons can be produced for all days and served through an API and dashboard, and frequency-based prediction beats a majority-class baseline.

1.7 Scope, Limitations and Assumptions

In scope: preprocessing of a 15-minute load series; construction of daily profiles; K-Means, DBSCAN, Ward agglomerative and Gaussian-mixture clustering; Isolation Forest, Local Outlier Factor, One-Class SVM and z-score anomaly detection; peak-day detection; rule-based behaviour labelling; frequency-based prediction of the label of a future date; a FastAPI backend and React dashboard; and integration with the team's applications through the dashboard shell and deep links.

Out of scope: load forecasting (the responsibility of the forecasting components); computing LIME or SHAP explanations (the XAI component); analysis of individual consumers or meters; real-time streaming of new readings; and deployment on a live utility system.

Assumptions and principal limitations: the analysed series (synthetic_realistic_load.csv) is treated as a stand-in for Sri Lankan utility demand, although its file name and its statistical regularities indicate that it is synthetic and its provenance is not documented; no verified list of real abnormal events is available, so detection accuracy cannot be measured directly; and the thresholds that define peak and low-demand days are relative to the same data they label. These and other limitations are examined in Section 7.5.

1.8 Individual Contribution

The author's individual contributions are summarised in Table 1.3. All of the work listed was carried out by the author within the behaviour-intelligence folder of the team repository.

Table 1.3: Individual contributions of the author

Area	Contribution
Data pipeline	Data loader with explicit columns, multi-stage cleaning, 15-minute resampling and construction of 1,977 daily 96-slot profiles; replacement of an earlier dataset after diagnosing that it had no temporal structure
Clustering	K-Means with silhouette-based selection of k over 2–12; DBSCAN with a data-driven eps from the k-distance knee; honest reporting of DBSCAN as degenerate
Anomaly and peak detection	Isolation Forest detector with z-score diagnostics; three-condition peak-day rule with reason text
Behaviour labelling	Priority rules producing four labels, risk levels and a written reason for every day
Model comparison	Clustering comparison of four algorithms ranked by mean rank over three indices; anomaly-detector comparison of four methods ranked by bootstrap stability, with a Spearman rank-agreement diagnostic and result caching
Prediction	Future-behaviour prediction by month and day-of-week matching with confidence and probability distribution
Service and interface	FastAPI backend with 12 endpoints and CSV report export; React/Vite dashboard, model-comparison and anomaly-comparison pages, prediction panel
Integration	Integrated sidebar shell embedding the three teammate applications with reachability checks; deep links from abnormal days to the XAI LIME page
Evaluation	Reproducibility check, API smoke test with latency measurement, back-test of the prediction module, and a critical audit of the component's limitations

1.9 Thesis Structure

Chapter 2 reviews the literature on demand forecasting, load-pattern clustering, anomaly detection and unsupervised model selection, and synthesises the research gap. Chapter 3 describes the research design, the data, the analysis pipeline, the model-comparison procedures and the evaluation measures. Chapter 4 presents the requirements and the design of the component within the complete solution. Chapter 5 describes the implementation, integration and testing. Chapter 6 reports the results, and Chapter 7 discusses them, including contributions, limitations and implications. Chapter 8 addresses commercialisation and sustainability, and Chapter 9 concludes

the thesis and proposes future directions. References, a glossary and appendices follow.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review is a targeted narrative review intended to position the component, rather than an exhaustive systematic survey. It began from the literature survey carried out for the project proposal, which was organised around electricity demand forecasting, clustering of electricity load patterns, electricity consumption behaviour and anomaly detection in electricity consumption, with priority given to studies of Sri Lankan demand. During implementation the review was extended to the methodological sources needed to justify the design: the original descriptions of the clustering and anomaly-detection algorithms, the internal cluster-validity indices, and work on assessing the stability of unsupervised results. [Confirm the databases searched (for example IEEE Xplore, ScienceDirect, Google Scholar), the date range and the number of papers screened.]

Statements about the applied studies in this chapter are limited to what those studies report, as recorded in the project proposal. Their numerical results are not compared with the results of this thesis, because they use different data, spatial levels and tasks.

2.2 Domain and Theoretical Foundations

2.2.1 Daily load profiles and demand behaviour

Electricity demand varies with the time of day, the day of the week, the season, public and religious holidays such as Poya days, and the weather. A convenient unit for studying this variation is the *daily load profile*: the sequence of demand values recorded over one day, here 96 values at 15-minute spacing. Two profiles can differ in level (how much demand there is) and in shape (when it occurs). Grouping days with similar profiles reveals recurring regimes, such as working days and weekends, while days that resemble no group are candidates for abnormal behaviour [13], [10]. Profiles are high-dimensional and their neighbouring values are strongly correlated, which affects the behaviour of distance-based algorithms and of validity indices.

2.2.2 Clustering methods

K-Means partitions observations into k groups so as to minimise the within-cluster sum of squared distances to the cluster means [14]. It is fast and widely used for load-pattern grouping, but it requires k in advance and favours compact, similarly sized clusters [15], [13]. **Agglomerative clustering with Ward linkage** builds a hierarchy by repeatedly merging the two clusters whose merger least increases the within-cluster variance [16]; it optimises a similar objective to K-Means without random initialisation. A **Gaussian mixture model** (GMM) represents the data as a weighted sum of Gaussian components and assigns each observation to its most probable component; with full covariance matrices it can fit elongated clusters, but in high dimensions it may devote a component to a small group of unusual points. **DBSCAN** groups points that lie in dense regions, defined by a radius eps and a minimum number of neighbours, and labels points in sparse regions as noise [17]. It does not need k and can find clusters of arbitrary shape, but it is sensitive to eps, and a single eps cannot separate clusters of different density.

2.2.3 Internal cluster validity

Without labels, cluster quality is judged with internal indices. The **Silhouette coefficient** compares, for every point, the mean distance to its own cluster with the mean distance to the nearest other cluster, giving a value in $[-1, 1]$ where higher is better [18]. The **Davies–Bouldin index** averages, over clusters, the worst ratio of within-cluster scatter to between-cluster separation, so lower is better [19]. The **Calinski–Harabasz score** is the ratio of between-cluster to within-cluster dispersion, adjusted for the number of clusters and points, so higher is better [20]. The indices encode different notions of a good partition and can disagree; in particular, the Silhouette score can be high when one large cluster is separated from a handful of extreme points. The **elbow method** looks for the k beyond which the within-cluster sum of squares (inertia) stops falling sharply, and knee-detection methods locate such bends automatically [21].

2.2.4 Unsupervised anomaly detection

Anomaly detection identifies observations that deviate from the expected pattern [22]. **Isolation Forest** isolates points by random axis-parallel splits; anomalies are

separated in fewer splits, so the average path length over many random trees gives an anomaly score [23]. **Local Outlier Factor** (LOF) compares the local density around a point with the densities around its neighbours, flagging points in relatively sparse neighbourhoods [24]. **One-Class SVM** learns a boundary that encloses most of the data in a kernel feature space, with the parameter nu bounding the fraction of points outside it [25]. The **z-score** rule flags values more than a fixed number of standard deviations from the mean; it is simple and transparent but assumes an approximately symmetric distribution and looks at one variable at a time. Isolation Forest, LOF and One-Class SVM are usually given an expected anomaly fraction (contamination or nu), which fixes in advance how many points they flag.

2.2.5 Selecting unsupervised models without ground truth

Because the number of flagged points is set by the contamination parameter, comparing detectors by how many anomalies they report says little about their quality. One alternative is to measure *stability*: a method whose output changes little when the data are resampled is more trustworthy than one whose output depends on which observations happen to be present. Hennig proposed assessing cluster stability by comparing clusterings of bootstrap resamples with the original clustering using the Jaccard coefficient [26]. The same idea can be applied to the set of days that a detector flags. Agreement between methods based on different principles provides further, independent evidence that the flagged points form a genuine group.

2.3 Related Research

2.3.1 Demand forecasting in Sri Lanka

A study of peak electricity demand in Sri Lanka compared multiple regression analysis (MRA) with an artificial neural network using CEB data and reported a mean absolute percentage error of 2.50% for the neural network against 4.72% for MRA [3]. A meta-analysis of forecasting in Australia, Pakistan and Sri Lanka found that machine-learning and hybrid methods generally outperform statistical models, but that many models do not adequately include external factors such as weather and behaviour [5]. Short-term forecasting studies for Sri Lanka have compared statistical and deep-learning models [2], and a broad review of forecasting methodologies identified interpretability and behavioural analysis as weaknesses of current models

[4]. Pattern-based statistical methods have also been used to analyse and forecast monthly demand [27]. These studies motivate the project as a whole; none of them describes the behaviour of individual days.

2.3.2 Clustering of load patterns

K-Means has been applied to classify load patterns of the Indonesian Jawa-Bali system by their peak-load characteristics, revealing variation associated with national holidays and changes in social activity [10]. Unsupervised methods including PAM, k-shape and DBA with dynamic time warping have been used to group industrial demand profiles and generate representative profiles for demand-side management [11]. Chicco reviewed and compared clustering methods for electrical load pattern grouping and the indices used to evaluate them [13]. Other work clusters households by consumption behaviour before forecasting at cluster level [6], or clusters smart-meter time series with K-Means and dynamic time warping and then applies ARIMA, NARX and DSHW models per cluster [7]; in both cases cluster-level forecasting reduced error. At the consumer level, spectral clustering and hidden Markov models have been used to identify user behaviour groups for marketing and energy-saving programmes [28].

2.3.3 Anomaly detection in electricity consumption

A hybrid framework combining unsupervised SR-CNN and martingale-based detection with a supervised boosted decision tree was applied to smart-meter data from about 4,300 Irish households and reported 90% accuracy and an F1-score of 0.894 in identifying suspicious consumption [8]. A VAE-DBSCAN method extracted features with a variational autoencoder and used DBSCAN to remove abnormal readings before ARIMA forecasting [12]. A framework using symbolic aggregate approximation with CART and MLP classifiers detected abnormal consumption in building data and gave energy managers interpretable tools [9]. These frameworks perform well on their tasks, but they operate on meters or buildings and on individual readings, and several depend on labelled anomalies.

Table 2.1: Summary of selected related studies and their relationship to this work

Study	Approach	Level / data	Relationship to this thesis
[3]	MRA vs ANN peak-demand prediction	Sri Lanka, CEB	Local evidence for ML; numbers only, no behaviour
[5]	Meta-analysis of forecasting methods	Australia, Pakistan, Sri Lanka	Notes missing behavioural factors
[10]	K-Means on load patterns	Jawa-Bali system	Closest to day-level clustering; no anomaly or labels
[11]	PAM, k-shape, DBA with DTW	Industrial plants	Profiling, not daily system behaviour
[6], [7]	Clustering before forecasting	Households, smart meters	Clustering only as a forecasting aid
[8]	SR-CNN, martingale + boosted trees	~4,300 households	Meter-level fraud; needs labels
[12]	VAE + DBSCAN	Consumption points	Anomalies removed for forecasting
[9]	SAX + CART/MLP	Buildings	Interpretable but building-level

Source: Author's summary of the studies reviewed in the project proposal.

2.4 Existing Systems, Products and Techniques

Utilities typically rely on forecasting software and supervisory control and data acquisition (SCADA) systems that report and predict load but do not classify days into behavioural categories. Analysts who want such categories must combine general-purpose tools: clustering and anomaly-detection libraries such as scikit-learn [29], and spreadsheet or business-intelligence dashboards. Table 2.2 compares the main technique families against the needs of this component.

Table 2.2: Comparison of technique families for day-level behaviour analysis

Family	Strengths	Limitations for this component
Calendar rules (weekday, weekend, holiday lists)	Transparent; no training	Cannot find unexpected behaviour; needs complete holiday calendars
Partitioning and hierarchical clustering (K-Means, Ward)	Fast; compact, interpretable groups; well-known indices	Need k; forced to assign every day, including abnormal ones

Family	Strengths	Limitations for this component
Density and mixture models (DBSCAN, GMM)	Can mark noise or soft membership	Sensitive to parameters; may isolate a few outliers instead of segmenting
Statistical thresholds (z-score, IQR)	Simple, explainable	One variable at a time; assume symmetric distributions
Isolation-, density- and boundary-based detectors (IF, LOF, OC-SVM)	Work on whole profiles; give continuous scores	Contamination fixed in advance; no explanation by themselves
Supervised classifiers	Measurable accuracy	Require labelled abnormal days, which are not available

2.5 Gap Synthesis and Conceptual Framework

The review shows that day-level clustering, anomaly detection and peak analysis have each been applied to electricity data, but separately, at other levels, or as aids to forecasting. It also shows that unsupervised methods are hard to compare and that the usual single-index or count-based comparisons can mislead. Table 2.3 links each gap to the objective and design element of this thesis that responds to it.

Table 2.3: Mapping of research gaps to objectives and design elements

Gap	Objective	Design element
No day-level behaviour description for operators	3, 5	Daily profiles, clusters and four readable labels with reasons
Anomaly detection only at meter level	4	Isolation Forest on whole daily profiles of system demand
Clustering only as a forecasting aid	3	Clustering used to describe demand regimes directly
Single-index cluster selection	3	Mean rank over Silhouette, Davies–Bouldin and Calinski–Harabasz
Detector comparison by anomaly count	4	Bootstrap Jaccard stability and cross-method agreement
Outputs not usable by other services	5	REST API, dashboard and deep links to XAI

The resulting conceptual framework treats behaviour intelligence as a pipeline: raw readings are cleaned and reshaped into daily profiles; unsupervised models describe each day from several angles (group, abnormality, peak magnitude); explicit rules

combine these descriptions into a label, a risk level and a reason; and the results are served to users and to other services. The framework is realised in the methodology of Chapter 3 and the design of Chapter 4.

3 RESEARCH METHODOLOGY

3.1 Research Design

The study follows Design Science Research (DSR), in which an artefact is built and evaluated to solve an identified problem [30]. The artefact is the Behaviour Intelligence component. The six activities of the DSR process model of Peffers et al. [31] were applied as shown in Figure 3.1: the problem was identified from the literature and the proposal (Chapter 1), objectives and requirements were defined (Chapters 1 and 4), the pipeline, API and dashboard were designed and developed (Chapters 4 and 5), the artefact was demonstrated through its dashboard and comparison pages (Chapters 5 and 6), it was evaluated (Chapter 6), and the work is communicated through this thesis.

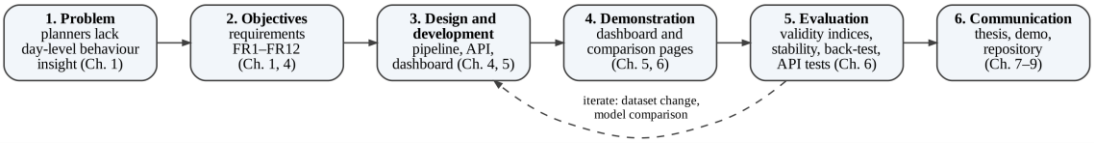


Figure 3.1: Design Science Research activities applied in this study (original in colour)

Source: Author's design, after the DSR process model [31].

The design–evaluation loop was iterated. The first version of the pipeline (Progress Presentation 1) was evaluated on an earlier dataset and gave clustering quality close to random; diagnosing that result led to replacing the dataset (Section 3.2.2). A second evaluation showed that single-index cluster selection and count-based detector comparison were misleading, which led to the model-comparison procedures of Section 3.3.7.

Evaluation is quantitative and largely internal, because the problem is unsupervised and no labelled abnormal or peak days exist. Clustering is evaluated with three validity indices; anomaly detection with bootstrap stability and agreement between methods; the labels by their composition across days of the week, months and years; the future-behaviour prediction with a temporal hold-out; and the software with reproducibility checks, an API test suite and latency measurements.

3.2 Data, Participants or Study Materials

No human participants were involved. The study material is a 15-minute electricity load series supplied in the component repository as **synthetic_realistic_load.csv**. Table 3.1 summarises it.

Table 3.1: Summary of the dataset used by the component

Item	Description
File	data/raw/synthetic_realistic_load.csv (7.54 MB)
Source	Supplied within the project repository. No generator script, source note or licence is recorded, and the original provenance is not documented [confirm the source of this file]
Period	1 January 2020 00:00 to 31 May 2025 00:00
Granularity	15 minutes; every step is 15 minutes; no duplicate timestamps; no missing values
Size	189,793 rows × 4 columns
Columns	Timestamp; Load Demand (kW); Temperature (°C); Solar Irradiance (W/m²)
Load range	629.73 to 4,365.20 kW; mean 1,747.81 kW; standard deviation 349.74 kW
Variables used	Timestamp and load only; temperature and irradiance are not used by the pipeline
Daily profiles	1,977 complete days (1,413 weekdays and 564 weekend days), 1 January 2020 to 30 May 2025

Source: Component dataset and data loader (data_loader.py); values measured by the evaluation script.

The proposal planned to use CEB data with calendar and holiday flags. That data was not obtained. The file actually used is described by its name as synthetic, and its properties (Section 3.2.3) are consistent with generated data. The results of this thesis therefore show how the component behaves on a realistic but artificial series; they do not show how it would behave on CEB measurements. This is treated as the principal threat to validity in Section 7.5.

3.2.1 Input variables

The loader reads only the timestamp and the load column, by explicit name. An earlier version detected the demand column automatically from keywords; on the first dataset it selected a per-capita energy column by mistake, which is why the column names are now fixed. Temperature and solar irradiance are present in the file

but are not read; their correlations with load are 0.155 and −0.243 respectively. No holiday, Poya-day or other calendar feature is used anywhere in the modelling.

3.2.2 Replacement of the earlier dataset

Until August 2026 the component used load_forecasting_dataset_corrected.csv (189,888 rows, 15 columns, same period). On that file K-Means reached a Silhouette score of only 0.0097, and every k from 2 to 10 scored below 0.01. A diagnosis recorded in the component's reports found that adjacent 15-minute load values were almost uncorrelated, that irradiance was non-zero at midnight and that temperature was identical at every hour, and concluded that the file contained no temporal structure for the methods to find. The dataset was therefore replaced. Table 3.2 compares two structural diagnostics on both files.

Table 3.2: Structural diagnostics of the earlier and current datasets

Diagnostic	Earlier dataset	Dataset in use
Mean correlation between adjacent 15-minute slots	0.003	0.6217
Principal components for 90% of profile variance	83	33
Best K-Means Silhouette score	0.0097	0.4729

Source: Component report model_comparison.md and evaluation script; saved metrics files.

3.2.3 Structure of the data

The series in use has clear but simple structure. Weekdays are flat and nearly identical, with mean demand of 1,834 to 1,840 kW from Monday to Friday, against 1,508 and 1,509 kW on Saturday and Sunday. There is a smooth annual cycle, with monthly mean load rising from 1,609 kW in December to 1,890 kW in June, and no growth trend: yearly means lie between 1,745 and 1,752 kW from 2020 to 2025. The average day has one evening peak at 19:30. Events known to have affected Sri Lankan demand, such as the COVID-19 restrictions of 2020–21, the 2022 load-shedding period and the Sinhala and Tamil New Year, leave no trace in these statistics, which supports the conclusion that the series is generated.

3.2.4 Preprocessing

Preprocessing follows the steps in Table 3.3. Two outlier rules are applied. The first removes extreme spikes above the median plus ten inter-quartile ranges and removed

nothing on this dataset. The second keeps only values inside the fences Q1 − 3×IQR and Q3 + 3×IQR, which here are 485.49 and 2,928.49 kW; it removed 691 readings (0.364%), all on the high side. Removed values are re-filled by interpolation when the series is resampled to a regular 15-minute grid.

Table 3.3: Preprocessing steps and their effect on the dataset

Step	Operation	Rows after	Effect
0	Load, parse timestamps, coerce demand to numeric, sort	189,793	–
1	Drop missing and negative demand	189,793	none removed
2	Drop spikes above median + 10×IQR (5,190.72 kW)	189,793	none removed
3	Linear interpolation, then forward and backward fill	189,793	nothing to fill
4	Keep values within [Q1 − 3×IQR, Q3 + 3×IQR] = [485.49, 2,928.49] kW	189,102	691 removed, all high
5	Resample to 15-minute means and interpolate gaps	189,793	gaps re-filled
6	Add date, hour and time slot (hour × 4 + minute ÷ 15)	189,793	–
7	Pivot to day × 96 slots, keep complete days	1,977 days	31 May 2025 dropped (1 slot)

Source: preprocessing.py and daily_profile.py; row counts from the evaluation script.

Step 4 has an important consequence. The 691 removed readings, between 2,928.78 and 4,365.20 kW, fall on 267 different days, and 56 of the 60 days later flagged as anomalous are among them. After cleaning, no daily peak exceeds 2,928.45 kW. The 3×IQR rule therefore removes genuine demand spikes, not only measurement errors, before peak and anomaly detection see the data. This is discussed as a limitation in Section 7.5.

3.3 Experimental or Study Procedure

The analysis is a five-stage pipeline (preprocessing and profiles, clustering, anomaly detection, peak detection, labelling) followed by two model-comparison procedures and a prediction module. Figure 3.2 shows the flow of data through the stages.

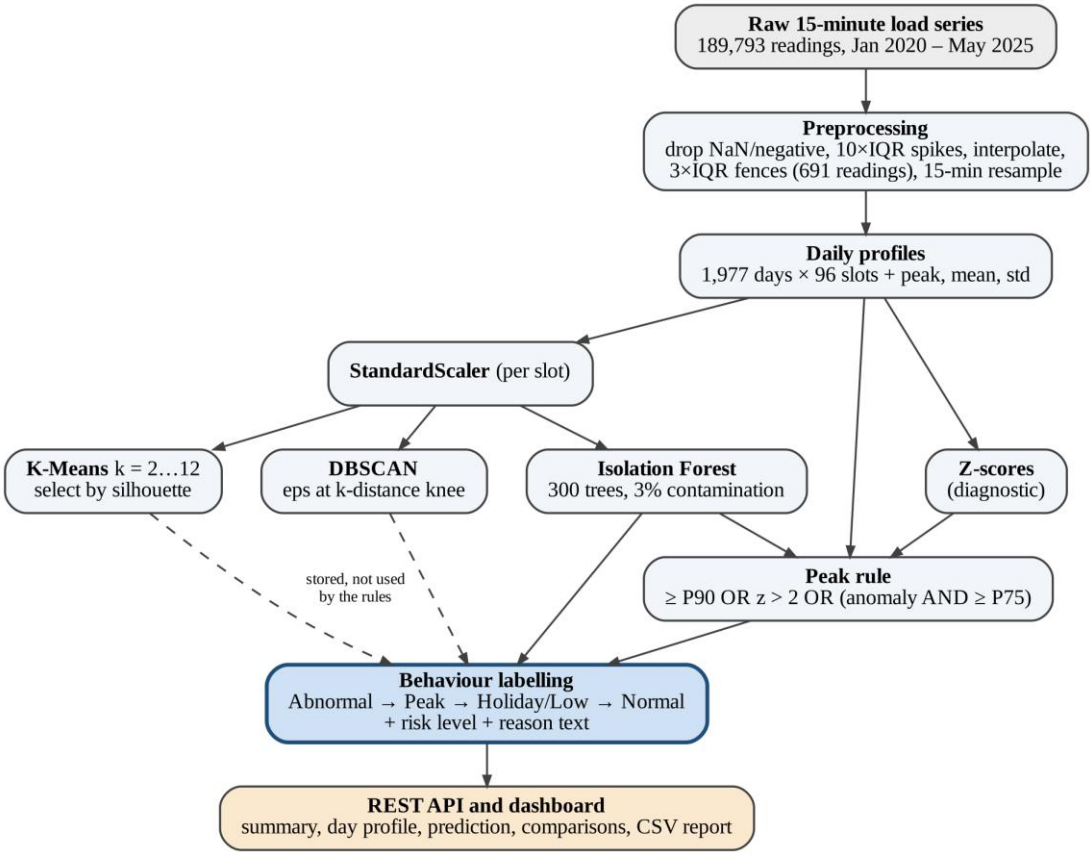


Figure 3.2: Analysis pipeline of the behaviour intelligence component (original in colour)

Source: Author's design, from the component code (app/ml).

3.3.1 Daily profiles and scaling

Each complete day becomes a row of 96 load values, p00 (00:00) to p95 (23:45), plus its peak (maximum), mean and standard deviation. A StandardScaler is fitted on the 1,977 × 96 matrix, so that each time slot is standardised across days, and saved for reuse. Scaling per slot means that clusters are formed on the shape and relative level of profiles rather than being dominated by the evening peak slots, which have the largest variance in kW.

3.3.2 Clustering and choice of k

K-Means (n_init = 20, random_state = 42) is fitted for every k from 2 to 12 on the scaled profiles, and the k with the highest Silhouette score is kept. The selected model, its cluster labels and the Silhouette and Davies–Bouldin scores are saved. The proposal also mentioned the elbow method; the implemented selection uses the

Silhouette score, and the inertia curve was computed during evaluation for comparison (Section 6.1.2).

3.3.3 DBSCAN with a data-driven radius

DBSCAN uses min_samples = 8. Its radius eps is set at run time to the knee of the sorted distance from each day to its eighth-nearest neighbour, located as the point of maximum perpendicular distance from the chord joining the ends of the normalised curve [21]. This replaced an earlier loop that increased eps in steps of 0.4 until any point escaped the noise label; that loop had silently stopped at eps = 2.80 and reported a 21-day cluster as if it were meaningful. The knee rule is reproducible and is reported honestly even when it yields a single cluster.

3.3.4 Anomaly detection

An Isolation Forest with 300 trees, contamination 0.03 and random_state = 42 is fitted on the scaled profiles [23]. Its anomaly score is the negated decision function, so higher means more anomalous and days with a positive score are flagged. Z-scores of the daily peak and mean are also computed and stored. An earlier version flagged a day if either Isolation Forest or the z-score (threshold 3.0) fired; the final flag uses Isolation Forest alone, because on this left-skewed data the largest absolute z-scores are 2.501 (peak) and 2.932 (mean), so the z-score rule cannot fire at 3.0 and only adds noise at lower thresholds (0, 2 and 76 days at thresholds 3.0, 2.5 and 2.0). The value 0.03 for contamination is not justified in the code; it fixes the flagged share at about 3% (60 of 1,977 days).

3.3.5 Peak-day detection

A day is flagged as a peak-demand day if any of three conditions holds: its peak demand is at or above the 90th percentile of daily peaks (2,884.67 kW); its peak z-score exceeds 2.0; or it is anomalous and its peak is at or above the 75th percentile (2,819.01 kW). The condition that fired is recorded as text. Because the stored peak z-score is an absolute value, the second condition also fires for days whose peak is far *below* the mean; the effect of this is quantified in Section 6.1.8.

3.3.6 Behaviour labelling

Labels are assigned by ordered rules, first match wins, as shown in Figure 3.3 and Table 3.4. Each label carries a fixed risk level, and each day receives a reason string built from the values that triggered the rule, for example *Flagged as anomaly (iso_score=0.130, peak_zscore=0.03) but not a peak day*.

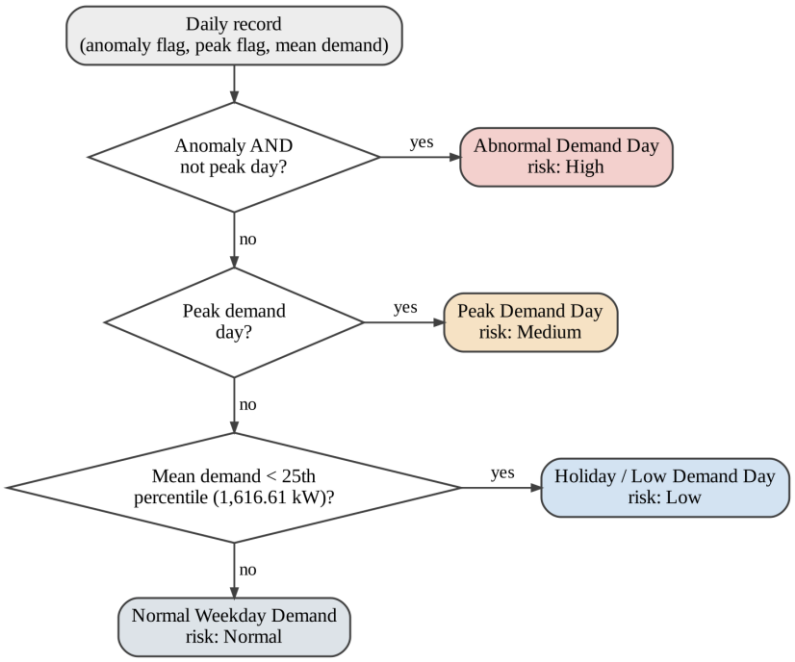


Figure 3.3: Ordered rules that assign the behaviour label (original in colour)

Source: Author's design, from behavior_labeling.py.

Table 3.4: Behaviour labelling rules

Priority	Condition	Label	Risk
1	Anomalous and not a peak day	Abnormal Demand Day	High
2	Peak demand day	Peak Demand Day	Medium
3	Mean demand below the 25th percentile of daily means (1,616.61 kW)	Holiday / Low Demand Day	Low
4	Otherwise	Normal Weekday Demand	Normal

Source: behavior_labeling.py.

Three properties of the rules matter for interpretation. First, the K-Means and DBSCAN clusters are stored with each day but are not inputs to the rules; the proposal had planned to use cluster assignment and calendar context. Second, an anomalous day that is also a peak day is labelled Peak, not Abnormal, so that only

anomalies without high demand reach the highest risk level. Third, the Holiday / Low label is a demand-level threshold; no holiday calendar is consulted.

3.3.7 Model-comparison procedures

Clustering. K-Means, Ward agglomerative clustering, a Gaussian mixture (full covariance, random_state = 42) and DBSCAN are fitted on the same scaled profiles, with k for the first three taken from the K-Means selection and DBSCAN using the knee eps. Each model is ranked on the Silhouette (higher better), Davies–Bouldin (lower better) and Calinski–Harabasz (higher better) indices, the three ranks are averaged, and the model with the lowest mean rank is selected. A model with fewer than two clusters has undefined indices; it is reported as not applicable rather than given zero scores, which would misrepresent it.

Anomaly detection. Isolation Forest, LOF (20 neighbours, contamination 0.03), One-Class SVM (RBF kernel, nu = 0.03) and the z-score rule are compared. Because three of them are tuned to the same 3% rate, the count of flagged days carries no information about quality. Instead, for each of 20 subsamples of 80% of the days (drawn without replacement, seeds 0–19), each detector is refitted and the Jaccard coefficient between its flagged set and its full-data flagged set is computed, following the stability idea of Hennig [26]. Detectors are ranked by mean Jaccard. The z-score rule is excluded when it flags no days, because a stability of zero would describe a rule that never fires, not an unstable one. As a diagnostic, Spearman rank correlations between the detectors' continuous scores are also reported.

Both comparisons generate a written explanation of the ranking, and their results are cached to JSON and recomputed only when the daily-profile file changes.

3.3.8 Future-behaviour prediction

For a requested date, the module finds historical days with the same month and day of week, falls back to the same month, and finally to all days if no match exists. The predicted label is the most frequent label among the matched days, the confidence is its share, and the full distribution over the four labels is returned with an explanation. The method is a transparent frequency baseline, not a forecasting model.

3.4 Measures and Evaluation Metrics

Table 3.5: Evaluation measures used in this study

Measure	Applied to	Interpretation
Silhouette coefficient [18]	Clustering	Separation of clusters, −1 to 1; higher is better
Davies–Bouldin index [19]	Clustering	Scatter relative to separation; lower is better
Calinski–Harabasz score [20]	Clustering	Between- to within-cluster dispersion; higher is better
Inertia	Choice of k	Within-cluster sum of squares; elbow view
Mean rank over the three indices	Clustering selection	Lower is better
Mean Jaccard over 20 subsamples [26]	Anomaly selection	Stability of the flagged set; ceiling 0.80 here (Section 6.1.6)
Mean symmetric difference	Anomaly selection	Days that differ per subsample; lower is better
Overlap counts and Spearman correlation	Anomaly detectors	Agreement between methods
Accuracy, precision, recall, F1, macro-F1	Prediction back-test	Against labels of a later hold-out period
Pass/fail, latency (median, min, max), run time, memory	Software	Verification and performance

Precision, recall and F1 for anomaly and peak detection, planned in the proposal against verified historical events, could not be computed, because no list of real abnormal or peak events exists for this series.

3.5 Data Analysis

All analysis was carried out in Python with pandas, NumPy and scikit-learn [29]. The pipeline functions of the component were run unchanged inside a separate scratch copy of the data, so that evaluation never altered the delivered results. The evaluation scripts recomputed every saved metric, compared the re-run outputs with the saved files day by day, produced the tables and figures of Chapter 6, and recorded all measured values in a machine-readable summary.

To evaluate the prediction module, which had no evaluation of its own, a temporal hold-out was used: history was restricted to days before 1 June 2024 (1,613 days)

and the module was asked to predict the label of each of the 364 later days. Predictions were compared with the labels assigned by the pipeline, and the majority-class rule (always predicting Normal Weekday Demand) was used as a baseline. Because the label thresholds were computed over the whole period, the test labels are not fully independent of the training period; this caveat applies to the reported accuracy.

3.6 Ethics, Privacy, Security and Responsible AI

The data are aggregated load values with no personally identifiable information, so no consumer can be identified and formal ethics clearance was not considered necessary [confirm against the SLIIT ethics requirement noted in the proposal]. Any future use of utility data must comply with Sri Lanka's Personal Data Protection Act, No. 9 of 2022 [32] and with the data owner's terms.

Responsible use of the outputs is addressed in three ways. Every label is accompanied by the rule and values that produced it, so that a planner can check it rather than trust it. The comparison pages state their methodology, including the fact that three detectors share a preset anomaly rate. And this thesis reports the limitations found during evaluation, including those that weaken the results, so that the labels are not over-relied upon. Security measures planned in the proposal (TLS and token-based authentication) are not yet implemented; the service runs on a local machine and its limits are described in Section 4.1.3.

3.7 Reproducibility and Research Artefacts

All stochastic steps use fixed seeds (K-Means, Gaussian mixture and Isolation Forest random_state = 42; bootstrap subsamples seeds 0–19); DBSCAN, Ward, LOF and the z-score are deterministic. Re-running the pipeline reproduced the saved behaviour labels, clusters, anomaly and peak flags with zero mismatches, and the clustering metrics and both comparison caches exactly, under two environments (scikit-learn 1.5.1 and 1.6.1). Table 3.6 summarises the reproducibility status.

Table 3.6: Reproducibility checklist and status

Item	Status
Code in version control	Repository branch IT22545176-Thanujan; work after 13 May 2026 still to be committed [commit before submission]
Fixed random seeds	Yes (Section 3.7)
Saved models and scaler	Yes: profile_scaler.pkl, kmeans_model.pkl, dbscan_model.pkl, isolation_forest_model.pkl
Saved intermediate results	Yes: cleaned data, daily profiles, clustering, anomaly, peak and label CSV files
Re-run reproduces saved results	Yes, 0 mismatches
Documented run commands	Yes (Section 5.5)
Pinned library versions match the environment	No: models were saved with scikit-learn 1.6.1, while requirements.txt pins 1.4.2
Dataset provenance documented	No [confirm the source of synthetic_realistic_load.csv]

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Stakeholders

The stakeholders identified in the proposal are: energy planners and system operators at CEB, who are the primary users of daily behaviour labels, anomaly reports and peak-day information; grid managers and utility staff, who monitor daily patterns and review flagged days; policymakers, regulators and researchers, who use aggregated behavioural insight for long-term planning; and the project team and future maintainers, who update models and thresholds. Within the project, the XAI and forecasting components are also consumers of the component's outputs.

4.1.2 Functional requirements

Table 4.1 lists the functional requirements of the proposal and the extent to which the implemented component satisfies them.

Table 4.1: Functional requirements and implementation status

FR	Requirement (from the proposal)	Status	Where
FR1	Accept 15-minute historical demand data with weather and calendar information	Partly: demand only; weather not used, no calendar data	data_loader.py
FR2	Clean missing values, implausible readings and outliers; standardise	Yes (spike removal is a limitation)	preprocessing.py
FR3	Aggregate into 96-point daily profiles	Yes, 1,977 days	daily_profile.py
FR4	Cluster profiles with K-Means and DBSCAN	Yes; DBSCAN degenerate on this data	clustering.py
FR5	Validate clusters with Silhouette and Davies–Bouldin	Yes, plus Calinski–Harabasz and mean-rank comparison	clustering.py, model_comparison.py
FR6	Detect anomalies with Isolation Forest and z-score	Yes; z-score kept as a diagnostic	anomaly_detection.py
FR7	Flag peak-demand days	Yes, with the absolute z-score issue (Section 6.1.8)	peak_detection.py
FR8	Assign four interpretable behaviour labels	Yes, with risk levels and reasons; no cluster or calendar input	behavior_labeling.py

FR	Requirement (from the proposal)	Status	Where
FR9	Produce insight outputs including calendar-based behaviour maps	Partly: summaries and distributions; no calendar map	analysis_service.py
FR10	Interactive visualisation dashboard	Yes	frontend/src
FR11	Expose outputs to Forecast, XAI and Scenario services	Partly: XAI deep link, CSV export, shared shell	App.jsx, routes.py
FR12	REST API endpoints	Yes, 12 routes	routes.py

Source: Project proposal, Table 2; status assessed against the component code.

4.1.3 Non-functional requirements

Table 4.2: Non-functional requirements and how they are addressed

Category	Requirement (summary)	How it is addressed
Performance	Process about 190,000 records within acceptable time	Full pipeline 4.9–8.9 s; read endpoints 70–160 ms; cached comparisons under 10 ms (Section 6.2)
Usability	Readable by planners without ML knowledge	Plain-language labels, risk badges, reason text, one-click prediction shortcuts
Interpretability	Users can see why a day received its label	Reason string per day; written explanation of every model selection
Reliability	No crashes on poor input; clear errors	Every endpoint returns a success flag and message; missing artefacts reported; complete days only
Maintainability	Modules replaceable independently	One module per pipeline stage; routes, controllers, services and ML layers separated
Scalability	Grow with more data and sources	Not yet addressed: single process, CSV files re-read per request, no pagination
Security	TLS and JWT; raw data not sent to the frontend	Not yet implemented: plain HTTP on localhost; CORS limited to the dashboard origin; one endpoint returns a day's 96 values
Compatibility	Python 3.9+; modern browsers	Runs on Python 3.12/3.13; React 18 build in any modern browser

Source: Project proposal, Table 3; assessment by the author.

4.2 Component Architecture within the Complete Solution

Within the complete system (Figure 1.1), the behaviour intelligence service sits in the AI layer beside the forecasting, XAI and scenario services. Each component is a separate application with its own FastAPI backend and React frontend, started

together by the team launcher. The behaviour intelligence frontend also serves as the shell of the whole system: its sidebar shows the component's own pages and opens the teammate applications in embedded views.

Figure 4.1 shows the internal architecture. The frontend, built with React and Vite, communicates with the backend over HTTP using JSON. The backend is layered: routes define the 12 endpoints; controllers wrap each operation in a uniform response envelope and error handling; a service layer answers dashboard and day-profile queries; and the ML package implements the pipeline, prediction and comparison logic. There is no database: inputs, intermediate results, models and caches are files in the data and models folders.

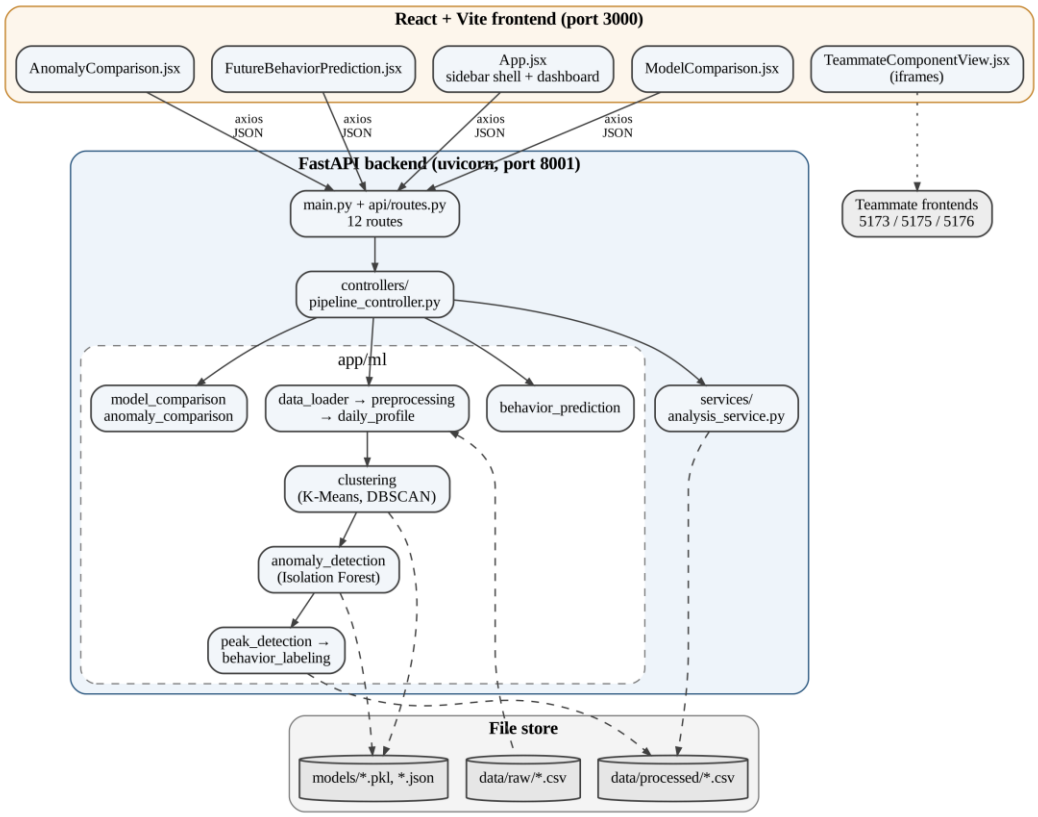


Figure 4.1: Internal architecture of the behaviour intelligence component (original in colour)

Source: Author's design, from the component code.

4.3 Design Alternatives and Decisions

Table 4.3 records the main design decisions, the alternatives that were rejected and the reasons, as recorded in the code and reports of the component.

Table 4.3: Key design decisions and justification

Decision	Alternative rejected	Reason
Rank clustering models by mean rank over three indices	Silhouette only	The Gaussian mixture has the highest Silhouette but the worst Davies–Bouldin and Calinski–Harabasz scores
Report undefined DBSCAN indices as not applicable	Report them as 0	Indices are undefined for one cluster, not zero
DBSCAN eps from the k-distance knee	Increase eps until any point leaves noise	The loop stopped at eps = 2.80 and reported a meaningless 21-day cluster
Final anomaly flag from Isolation Forest only	Isolation Forest OR z-score	A symmetric z-score cannot flag extremes in left-skewed columns
Rank detectors by bootstrap stability	Anomaly rate or consensus vote	Both reduce to the shared contamination setting
Exclude the z-score from ranking when it flags nothing	Give it stability 0	Zero would describe a rule that never fires, not instability
Read columns by explicit name	Keyword auto-detection	Auto-detection selected the wrong column on the first dataset
Cache comparison results	Recompute on every page load	The anomaly comparison takes about 15–35 seconds uncached
Replace the dataset	Keep the first dataset	It had no temporal or cross-variable structure
Embed teammate apps only after a reachability check	Plain iframe	Avoids a broken-frame error inside the shell
Store results as files	Database	Simplicity for a research prototype; a database is future work

Source: Code comments and reports in the component repository.

4.4 Detailed Design

4.4.1 Input representation

The unit of analysis is the day. Each day is represented by a vector of 96 load values at 15-minute spacing and by three summary statistics (peak, mean and standard deviation). For clustering and anomaly detection the vector is standardised per slot with the saved scaler; the summary statistics are used in raw kW by the peak rule and the labelling rules, so that thresholds can be stated in units planners understand.

4.4.2 Analysis modules

Table 4.4: Algorithms in the component and their configuration

Algorithm	Role	Configuration	How chosen
StandardScaler	Scaling	Defaults, fitted on 1,977 × 96 profiles	–
K-Means	Production clustering	k = 2, n_init = 20, random_state = 42	Max Silhouette over k = 2–12
DBSCAN	Production clustering (degenerate)	eps = 4.4183, min_samples = 8	Knee of the 8-distance curve
Ward agglomerative	Comparison only	k = 2, Ward linkage	k from K-Means
Gaussian mixture	Comparison only	2 components, full covariance, random_state = 42	k from K-Means
Isolation Forest	Production anomaly detector	300 trees, contamination 0.03, random_state = 42	Fixed values
Z-score	Diagnostic	|z| of peak and mean, threshold 3.0	Fixed
Local Outlier Factor	Comparison only	20 neighbours, contamination 0.03	Fixed
One-Class SVM	Comparison only	RBF kernel, nu = 0.03, gamma = scale	Fixed
Peak rule	Peak detection	P90 OR z > 2.0 OR (anomaly AND P75)	Fixed
Frequency matching	Future-date prediction	Month + weekday, then month, then all	Fixed fallback order

Source: clustering.py, anomaly_detection.py, model_comparison.py, anomaly_comparison.py, peak_detection.py, behavior_prediction.py.

4.4.3 Labelling and explanation

The labelling design favours transparency over sophistication. Each rule is a short condition on values the user can see on the dashboard (anomaly score, peak flag, mean demand), the order of the rules is fixed, and the reason string repeats the values that triggered the rule. This makes every label auditable: a planner who disagrees with a label can see exactly which number caused it. Deeper explanation of *why* a profile is anomalous is delegated to the XAI component, which receives the date through a deep link and computes a LIME explanation [33].

4.4.4 Model-comparison services

The two comparison modules are designed as read-only services that recompute on demand and cache their results. Each returns the per-model metrics, the ranks, the selected model, a generated paragraph explaining the selection, a list of supporting points and a comparison table, together with the time of computation and whether the result came from the cache. The anomaly comparison additionally returns the Spearman matrix, a note on the shared contamination setting, and the bootstrap configuration. The cache is invalidated when the modification time of the daily-profile file changes, so that the pages always describe the current data.

4.4.5 Service interface

Table 4.5 lists the 12 endpoints. Pipeline stages are exposed as POST endpoints that must be called in order, because each reads the file written by the previous stage; read endpoints are GET requests. All /api routes except the CSV download return the envelope {"success": bool, "message": str, "data": {...}}. Interactive documentation is generated by FastAPI at /docs.

Table 4.5: REST endpoints of the behaviour intelligence service

Method	Path	Purpose
GET	/health	Liveness check
POST	/api/generate-daily-profiles	Clean raw data and build daily profiles
POST	/api/run-clustering	Fit scaler, K-Means and DBSCAN; save models and metrics
POST	/api/run-anomaly-detection	Fit Isolation Forest and compute z-scores
POST	/api/detect-peak-days	Apply the peak rule
POST	/api/generate-behavior-labels	Apply the labelling rules
GET	/api/dashboard-summary	Counts, distributions, recent abnormal days and all days
GET	/api/get-day-profile?date=	96 points, label, risk and reason for one day
GET	/api/predict-future-behavior?date=	Likely label, confidence and distribution for a date
GET	/api/model-comparison	Clustering comparison (cached)
GET	/api/anomaly-comparison	Anomaly-detector comparison (cached)
GET	/api/download-report?label=	CSV report, optionally filtered by label

Source: main.py and api/routes.py.

4.4.6 User interface

The interface is a single-page application with a left sidebar. Under *Behaviour Intelligence* it offers the Dashboard, Model Comparison and Anomaly Comparison pages; under *Other Components* it opens the Demand Forecasting, Explainable AI and Adaptive Forecasting applications of the teammates. The dashboard is organised from summary to detail: five summary cards; label and risk distributions; the 15 most anomalous days and the anomaly-score trend of the last 180 days; a day explorer with a detail panel and the selected day's 96-point load profile; a table of recent abnormal days with LIME links; and a future-behaviour prediction panel. The screens are shown in Section 5.2.

4.5 Standards and Professional Practice

The design follows established practice for unsupervised learning: methods are compared on the same data under one protocol, several validity indices are reported rather than one, undefined results are reported as such rather than replaced by convenient numbers, random seeds are fixed, and results are reproducible from saved artefacts. Software practice follows the team workflow agreed in the proposal: version control on GitHub with a branch per component, a REST/JSON interface between frontend and backend, and separation of routes, controllers, services and ML code. The thesis follows the SLIIT dissertation guidelines for structure and formatting and uses IEEE referencing.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The component was developed on a Windows 11 laptop with Visual Studio Code and Jupyter notebooks. Table 5.1 lists the technologies used and why each was chosen. The versions shown are those of the environment that runs the backend; the saved models were written with scikit-learn 1.6.1 under a second (Anaconda) environment, and loading them under 1.5.1 produces a version warning but identical results.

Table 5.1: Technology stack and justification

Layer	Technology (version)	Justification
Language	Python 3.12.7 (backend); 3.13.5 (notebooks, evaluation)	Standard for data analysis; all required libraries available
Data handling	pandas 2.2, NumPy 1.26	Efficient cleaning, resampling and pivoting of 190,000 readings
Machine learning	scikit-learn 1.5.1 / 1.6.1 [29]	K-Means, DBSCAN, Ward, GMM, Isolation Forest, LOF, One-Class SVM and validity indices in one library
Statistics	SciPy 1.13	Spearman rank correlation
Backend	FastAPI 0.115, uvicorn 0.30, Pydantic 2.7	Fast REST APIs with automatic documentation; same framework as the teammates' services
Frontend	React 18.3, Vite 5.4, Recharts 2.15, axios 1.x	Component-based UI, fast builds, declarative charts
Persistence	CSV, JSON and pickle files	Simple, inspectable artefacts for a research prototype
Integration	Team launcher start-all.py	Starts all four backends and frontends with one command
Version control	Git and GitHub	Branch per component in the team repository

Source: requirements.txt, package.json and the installed environments.

5.2 Component Implementation

Backend and pipeline. The backend contains about 4,300 lines of Python. The ML package holds one module per stage (data_loader, preprocessing, daily_profile, clustering, anomaly_detection, peak_detection, behavior_labeling), plus behavior_prediction and the two comparison modules, which are the largest at 446 and 531 lines. pipeline_runner.py runs the five stages in one call. Each stage reads

the previous stage's CSV file and writes its own, so a stage can be re-run without repeating the earlier ones, and every intermediate result can be inspected. The controllers catch all exceptions and return them in the response envelope, and the service layer builds the dashboard summary and day-profile responses from the final labels file.

Frontend. The React frontend contains about 1,700 lines of JSX and 1,250 lines of CSS. App.jsx implements the sidebar shell and the dashboard; ModelComparison.jsx and AnomalyComparison.jsx implement the comparison pages; FutureBehaviorPrediction.jsx implements the prediction panel; and TeammateComponentView.jsx embeds a teammate application after checking within two seconds that it is reachable, otherwise showing the owner's name and start commands. Charts are drawn with Recharts and data are fetched with axios. The production build compiles 887 modules into a 640 kB JavaScript bundle (181 kB compressed).

Figure 5.1 to Figure 5.10 show the implemented interface.

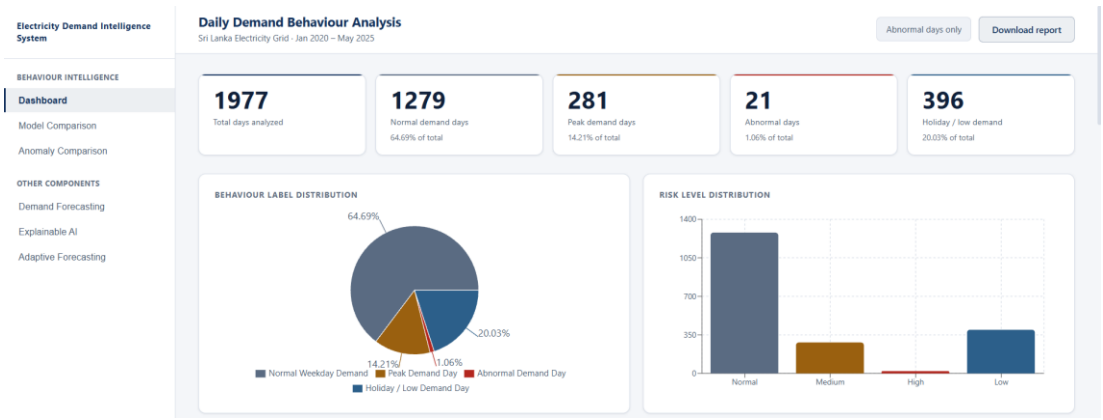


Figure 5.1: Dashboard: sidebar, summary cards, behaviour-label distribution and risk-level distribution (original in colour)

Source: Screenshot of the running component (localhost:3000).

The summary cards (Figure 5.1) give the number of days analysed and the count and share of each label. The pie chart and the risk chart repeat the distribution in two forms, because planners think in terms of both the kind of day and the level of attention it needs.

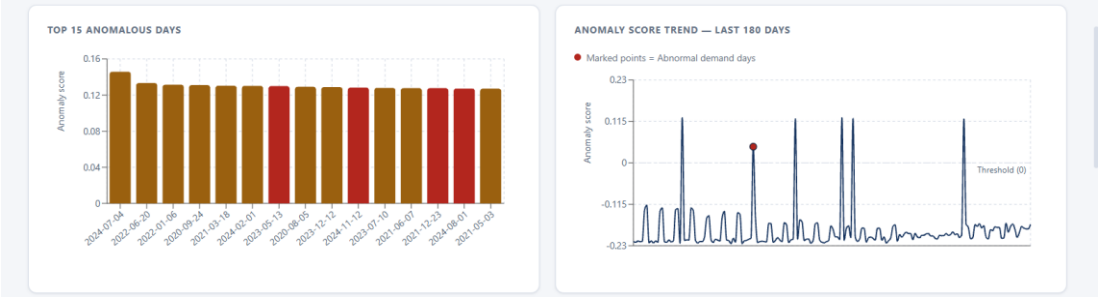


Figure 5.2: Dashboard: the 15 most anomalous days and the anomaly-score trend for the last 180 days (original in colour)

Source: Screenshot of the running component. Red bars and marked points are days labelled Abnormal Demand Day; amber bars are anomalous days that were labelled Peak Demand Day.

The anomaly charts (Figure 5.2) show the Isolation Forest score: the bar chart ranks the 15 highest-scoring days, coloured by their final label, and the line chart shows the score over the last 180 days with the decision threshold at zero.

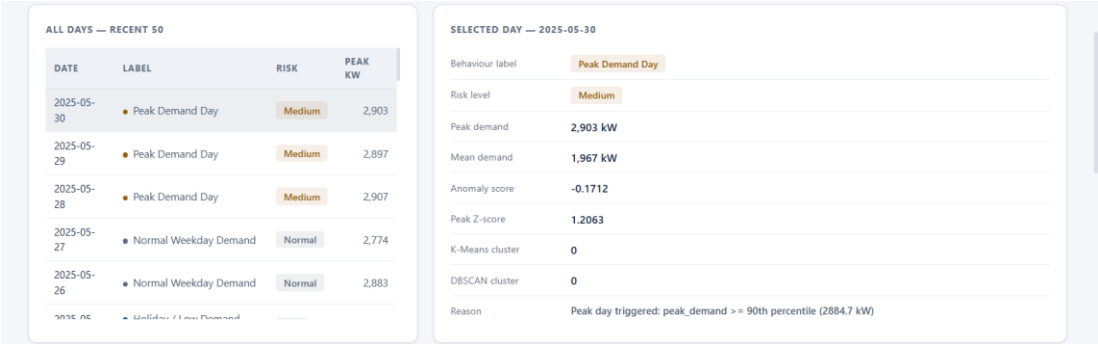


Figure 5.3: Dashboard: day explorer with the detail panel of a selected day (original in colour)

Source: Screenshot of the running component.

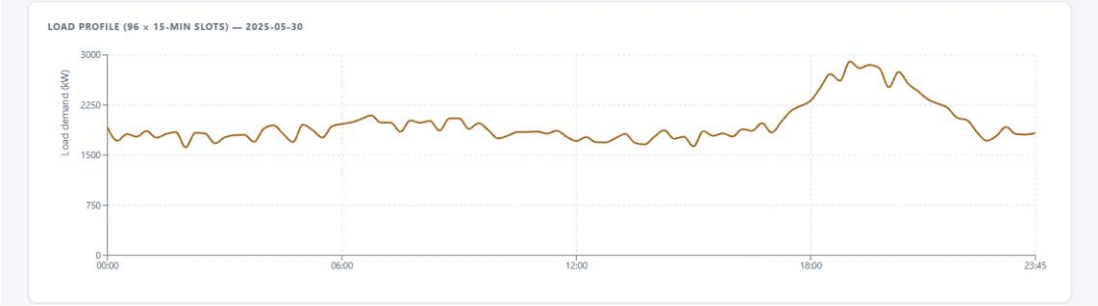


Figure 5.4: Dashboard: 96-point load profile of the selected day (original in colour)

Source: Screenshot of the running component.

The day explorer (Figure 5.3) lists the 50 most recent days. Selecting a day shows its label, risk, peak and mean demand, anomaly score, peak z-score, K-Means and DBSCAN clusters and the reason for its label; in the example, 30 May 2025 is a Peak Demand Day because its peak of 2,903 kW is above the 90th-percentile threshold of

2,884.7 kW. The load-profile chart (Figure 5.4) draws the selected day's 96 values, here with the evening peak around 19:00.



Figure 5.5: Dashboard: recent abnormal days with reasons and links to the LIME explanation in the XAI component (original in colour)

Source: Screenshot of the running component.

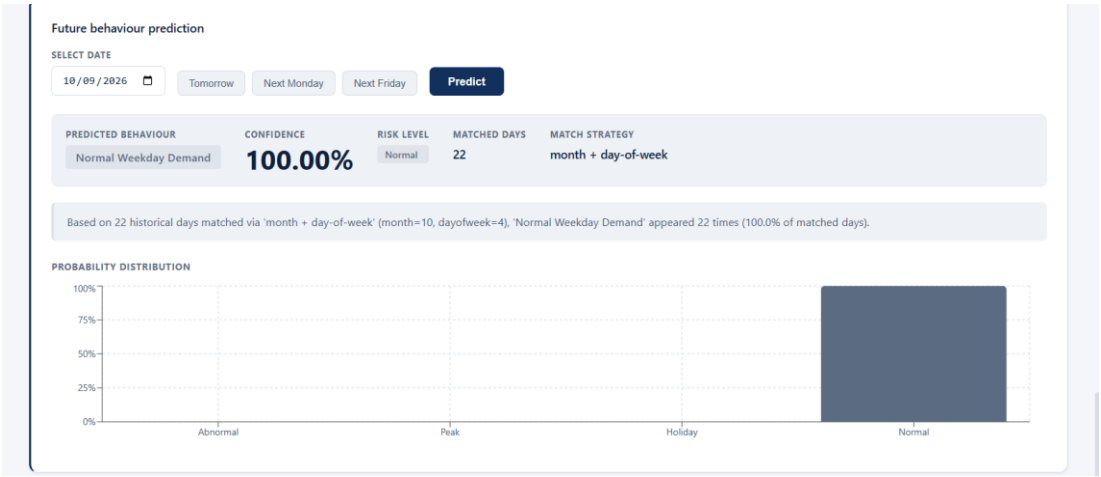
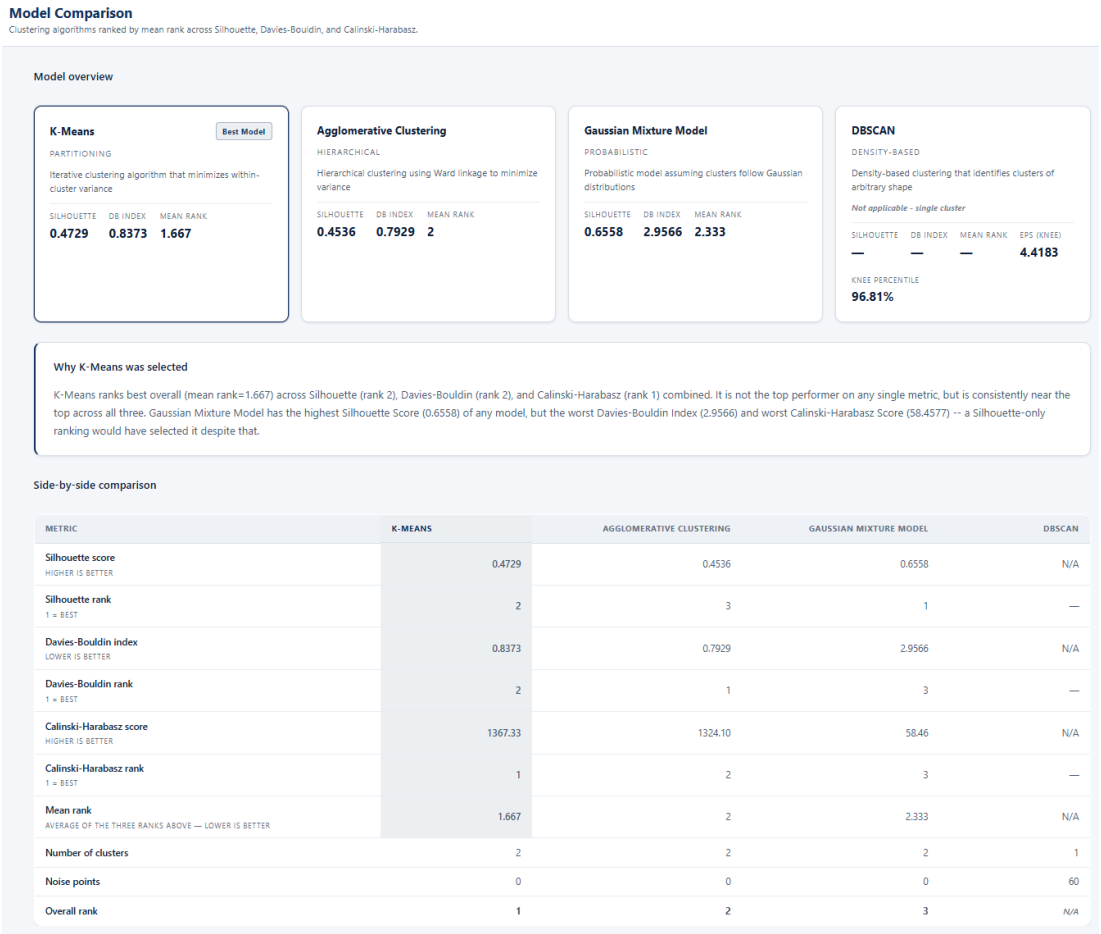


Figure 5.6: Dashboard: future-behaviour prediction for a selected date (original in colour)

Source: Screenshot of the running component.

The recent-abnormal-days table (Figure 5.5) lists the last ten Abnormal Demand Days with their scores and reasons and a LIME button that opens the XAI component's explanation for that date. The prediction panel (Figure 5.6) takes a date, or one of the Tomorrow, Next Monday and Next Friday shortcuts, and shows the predicted label, confidence, risk, number of matched historical days, the matching strategy, a sentence of explanation and the probability distribution over the four labels.



METRIC	K-MEANS	AGGLOMERATIVE CLUSTERING	GAUSSIAN MIXTURE MODEL	DBSCAN
Silhouette score HIGHER IS BETTER	0.4729	0.4536	0.6558	N/A
Silhouette rank 1 = BEST	2	3	1	—
Davies-Bouldin index LOWER IS BETTER	0.8373	0.7929	2.9566	N/A
Davies-Bouldin rank 1 = BEST	2	1	3	—
Calinski-Harabasz score HIGHER IS BETTER	1367.33	1324.10	58.46	N/A
Calinski-Harabasz rank 1 = BEST	1	2	3	—
Mean rank AVERAGE OF THE THREE RANKS ABOVE — LOWER IS BETTER	1.667	2	2.333	N/A
Number of clusters	2	2	2	1
Noise points	0	0	0	60
Overall rank	**1**	**2**	**3**	*N/A*

Figure 5.7: Model Comparison page: model cards, selection explanation and side-by-side metrics (original in colour)

Source: Screenshot of the running component.

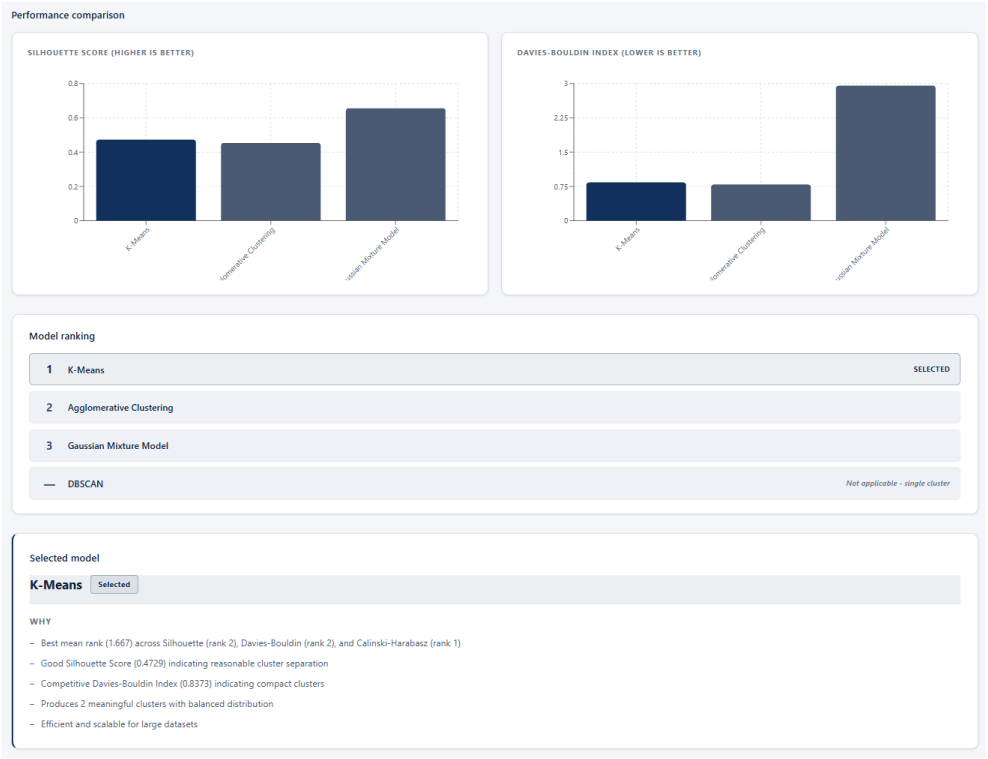


Figure 5.8: Model Comparison page: Silhouette and Davies–Bouldin charts, ranking and selected model (original in colour)

Source: Screenshot of the running component.

The Model Comparison page (Figure 5.7 and Figure 5.8) presents the four clustering algorithms as cards, explains why K-Means was selected, and compares all three indices and their ranks side by side, marking DBSCAN as not applicable because it forms a single cluster.

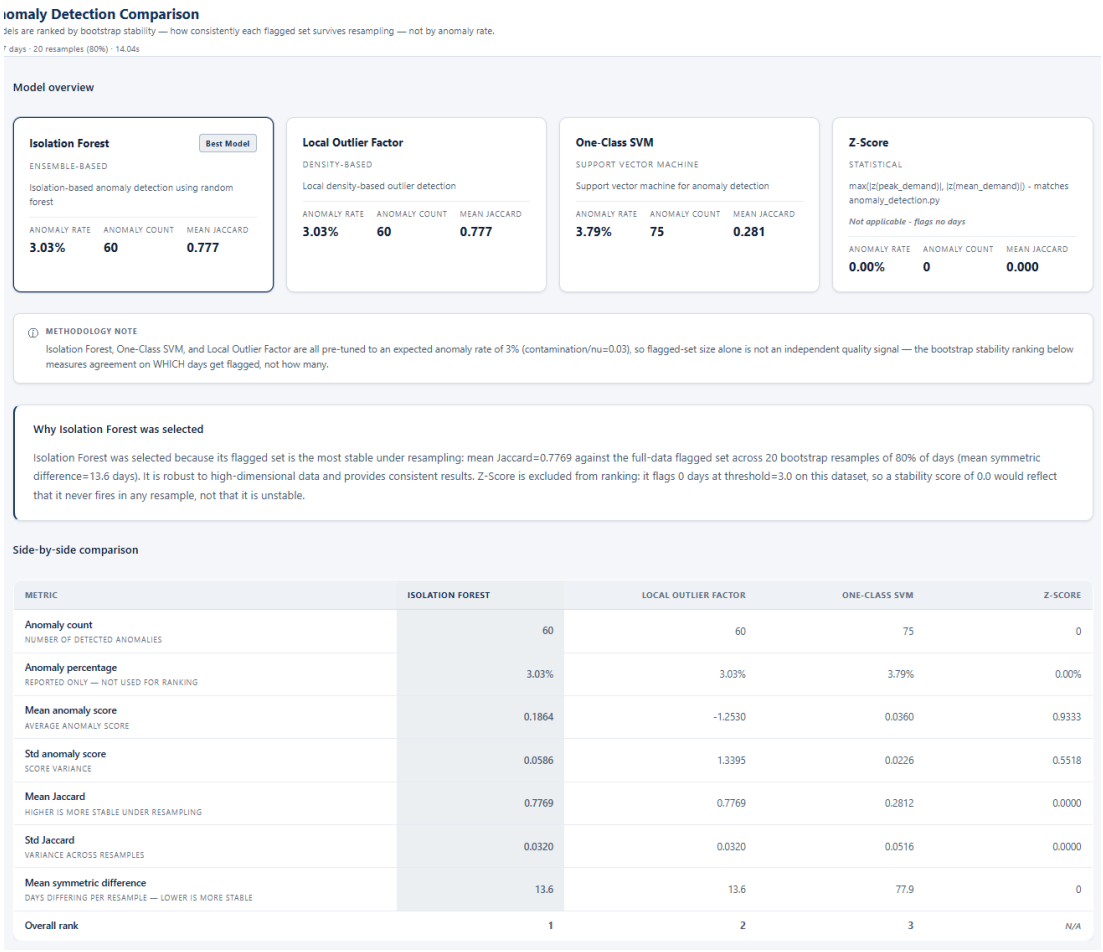


Figure 5.9: Anomaly Comparison page: detector cards, methodology note, selection explanation and side-by-side metrics (original in colour)

Source: Screenshot of the running component.

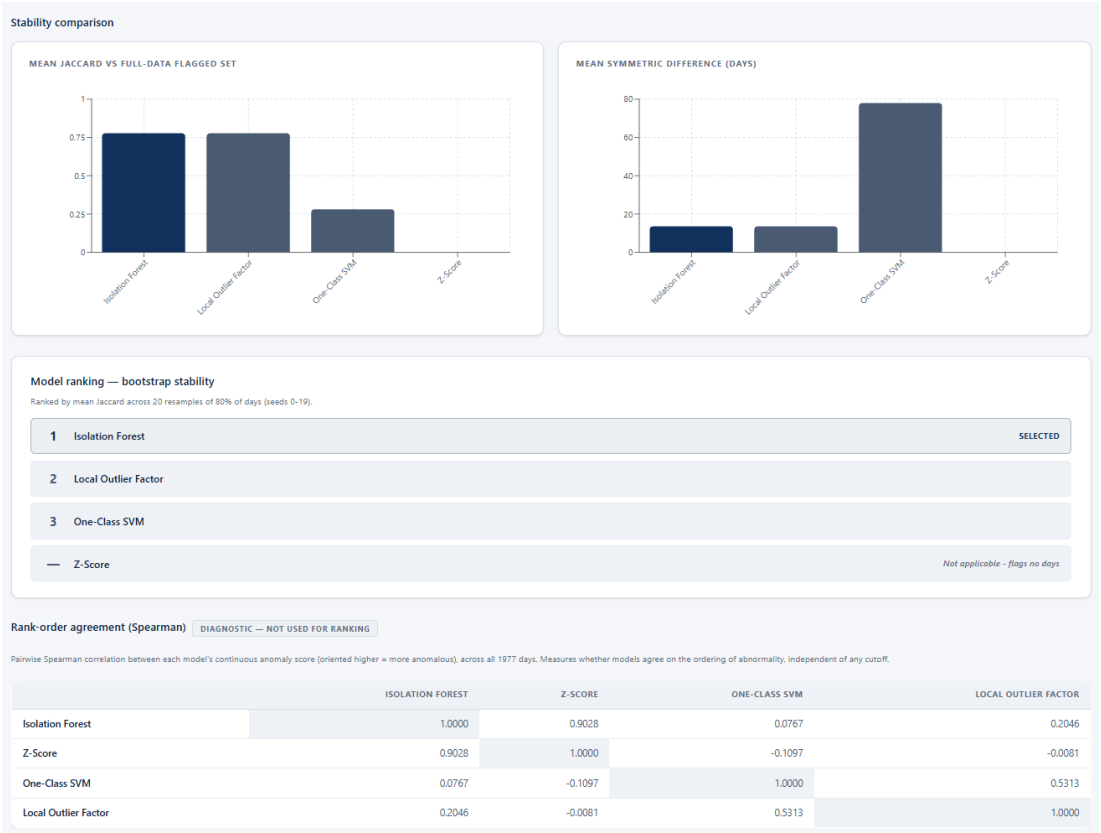


Figure 5.10: Anomaly Comparison page: stability charts, ranking and Spearman rank-agreement diagnostic (original in colour)

Source: Screenshot of the running component.

The Anomaly Comparison page (Figure 5.9 and Figure 5.10) shows the four detectors, a methodology note explaining why the shared 3% setting makes anomaly counts uninformative, the stability metrics and charts, the ranking, and the Spearman matrix, which is labelled as a diagnostic that is not used for ranking.

5.3 Integration with the Other Components

The four components are started together by the team launcher, start-all.py, which runs each backend and frontend on fixed ports, prints their addresses and stops all processes on exit (Table 5.2). It can also start a subset, for example only this component.

Table 5.2: Components started by the team launcher

Key	Component (owner)	Backend port	Frontend port
bi	Behaviour intelligence (Thanujan T)	8001	3000
xai	Explainable AI (Madushani R.P.)	8000	5173

Key	Component (owner)	Backend port	Frontend port
forecast	Demand forecasting (Dewmini D.M.D.P.)	8002	5175
nn	Adaptive forecasting (Janahan R.)	8003	5176

Source: start-all.py.

Integration is implemented through the user interface. The behaviour intelligence dashboard is the shell of the system: its sidebar opens the three teammate frontends in embedded views. Each abnormal day, and a predicted abnormal day, has a LIME button that opens the XAI component at /anomaly-diagnosis with source=bi and the date as parameters; the XAI page recognises the source, shows a Behaviour Intelligence banner and links back to the dashboard. The teammates' backends allow requests from the dashboard's origin. The CSV report endpoint gives other tools a simple export of all labels.

The integration has three gaps, reported in Section 6.4: the XAI component explains dates using the earlier dataset rather than the series this component now analyses; the XAI code expects a source=bi-predict link and a list of matched dates that this component does not send; and no other component yet calls this service's API, so behaviour labels are not passed to the forecasting services as context, as the proposal intended.

5.4 Verification and Software Testing

The repository contains no automated unit tests. Verification was therefore carried out with purpose-written evaluation scripts that call the component's own functions and endpoints against a scratch copy of the data, so that testing could not change the delivered results. Table 5.3 lists the checks.

Table 5.3: Verification checks and results

Check	Method	Result
Reproducibility of results	Re-run the full pipeline; compare with saved files day by day	0 mismatches in labels, risk levels, clusters, anomaly and peak flags; scores identical
Reproducibility of comparisons	Recompute both comparison modules	Identical to the saved caches
Cross-environment consistency	Run under scikit-learn 1.5.1 and 1.6.1	Identical results

Check	Method	Result
API integration	24 checks with FastAPI's test client	24 passed, 0 failed
Saved models	Load each pickle and print its parameters	All four load; parameters as expected
z-score thresholds	Diagnostic script at 3.0, 2.5, 2.0	0, 2 and 76 days flagged
Frontend build	Production build with Vite	Succeeds (887 modules)
Prediction accuracy	Temporal hold-out back-test	78.85% accuracy (Section 6.1.10)

Source: Evaluation scripts written for this thesis (thesis_evidence/scripts).

The API test suite covered: the health check; the published schema (12 paths); the five pipeline stages called in order with the exact expected counts; dashboard totals; day profiles for a valid date, an unknown date and a missing parameter (HTTP 422); predictions for a valid and an invalid date; both comparisons, uncached and cached; the CSV report for all days, a label filter and an unknown label; CORS acceptance of the dashboard origin and rejection of another origin; and the behaviour of the dashboard and report before the pipeline has run. The code also handles missing columns, non-numeric values, incomplete days, degenerate clusterings, corrupt caches and dates without historical matches.

The tests also revealed weaknesses that are recorded for future work: failures are returned with HTTP status 200 and a success flag rather than an error status; raw exception messages are passed to the client; an invalid date string is not validated at the route; and the anomaly stage fails with an unhelpful error if the saved scaler is missing.

5.5 Deployment and Operation

The component runs locally. The backend is started from the backend folder with py -3.12 -m uvicorn app.main:app --port 8001 and the frontend from the frontend folder with npm run dev, which serves the dashboard at http://localhost:3000. The whole system is started from the repository root with py -3.12 start-all.py, or start-all.py --only bi for this component alone. The Python 3.12 interpreter must be named explicitly on the development machine, because the default interpreter (3.14) does not have uvicorn installed.

The pipeline is re-run either by calling the five POST endpoints in order or by calling run_full_pipeline from the backend folder; a full run takes about 5 to 9 seconds and rewrites the processed files, models and clustering metrics. An older standalone script, train_models.py, is still referred to by the component README but has diverged from the API pipeline (different k range, the old DBSCAN loop and the old combined anomaly flag) and would produce a different set of results; it should not be used to reproduce the results in this thesis.

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Experimental setting

All results are for the 1,977 complete days from 1 January 2020 to 30 May 2025, each represented by 96 standardised 15-minute load values, as described in Chapter 3. All models are fitted and evaluated on all days, because the methods are unsupervised and there is no held-out labelled set; the only temporal hold-out is the back-test of the prediction module (Section 6.1.10). The numbers reported are those of the delivered result files, which were reproduced exactly by re-running the pipeline.

6.1.2 Choice of the number of clusters

Table 6.1 and Figure 6.1 show the three validity indices and the inertia of K-Means for k = 2 to 12. k = 2 is best on all three indices at once: it has the highest Silhouette (0.4729), the lowest Davies–Bouldin index (0.8373) and the highest Calinski–Harabasz score (1,367.33). From k = 6 upwards, K-Means begins to give single anomalous days a cluster of their own (smallest cluster of one day). The inertia curve bends most clearly at k = 3, so an elbow reading alone would suggest three clusters; the three indices, however, all prefer two.

Table 6.1: K-Means validity indices, inertia and smallest cluster for k = 2 to 12

k	Silhouette ↑	Davies–Bouldin ↓	Calinski–Harabasz ↑	Inertia	Smallest cluster (days)
2	**0.4729**	**0.8373**	**1,367.33**	112,149	581
3	0.3839	1.0281	1,095.25	89,963	433
4	0.2770	1.3785	807.33	85,202	336
5	0.3095	1.7812	650.98	81,791	249
6	0.3371	2.1552	566.75	77,856	1
7	0.1705	2.3790	503.33	74,928	35
8	0.2520	2.0107	448.17	73,185	1
9	0.1933	2.0667	398.33	72,461	1
10	0.2537	1.8219	363.62	71,250	1
11	0.2234	1.8350	336.17	70,037	1

k	Silhouette ↑	Davies–Bouldin ↓	Calinski–Harabasz ↑	Inertia	Smallest cluster (days)
12	0.1614	1.9742	309.79	69,414	1

Source: models/clustering_metrics.json (Silhouette) and evaluation script (other columns).

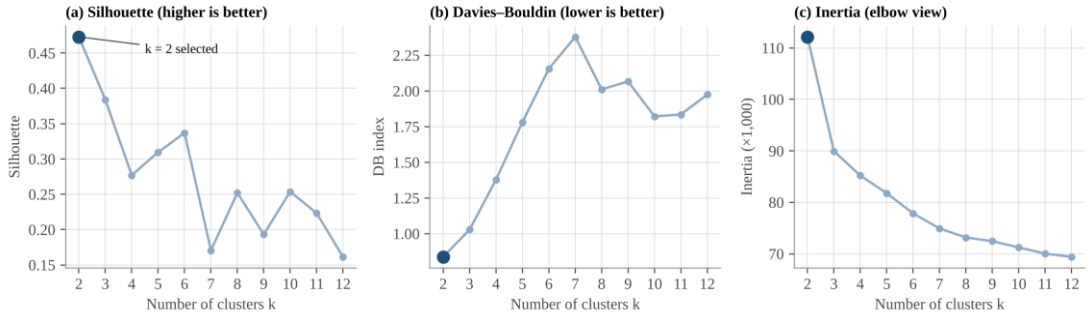


Figure 6.1: Silhouette, Davies–Bouldin and inertia of K-Means for k = 2 to 12 (original in colour)

Source: Author's analysis; values from Table 6.1.

6.1.3 Comparison of clustering algorithms

Table 6.2 and Figure 6.2 compare the four clustering algorithms at k = 2. K-Means has the best mean rank (1.667) and was selected. It is not best on any single index except Calinski–Harabasz: Ward agglomerative clustering has a slightly lower Davies–Bouldin index (0.7929), and the Gaussian mixture has by far the highest Silhouette (0.6558).

Table 6.2: Comparison of clustering algorithms on the daily profiles

Model	Silhouette	Davies–Bouldin	Calinski–Harabasz	Cluster sizes	Ranks (S/DB/CH)	Mean rank
K-Means	0.4729	0.8373	1,367.33	1,396 / 581	2 / 2 / 1	**1.667 (selected)**
Ward agglomerative	0.4536	0.7929	1,324.10	1,417 / 560	3 / 1 / 2	2.000
Gaussian mixture	0.6558	2.9566	58.46	1,940 / 37	1 / 3 / 3	2.333
DBSCAN (eps 4.4183)	undefined	undefined	undefined	1,917 + 60 noise	–	not ranked

Source: models/model_comparison_cache.json, reproduced by the evaluation script.

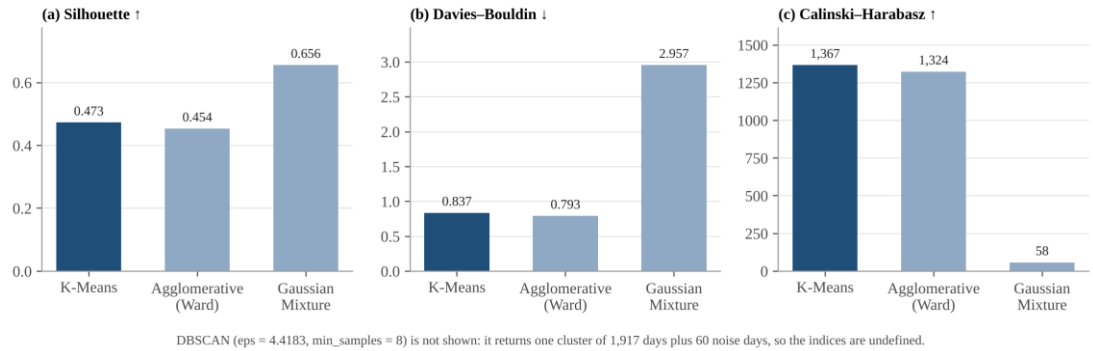


Figure 6.2: Validity indices of the clustering algorithms at k = 2 (original in colour)

Source: Author's analysis; values from Table 6.2.

The Gaussian mixture's high Silhouette illustrates why a single index was not used. Its two components are a large group of 1,940 days and a small group of 37 days, 36 of which are Isolation Forest anomalies. It is, in effect, separating a few extreme days from everything else, which produces a high Silhouette but a poor Davies–Bouldin index and a very low Calinski–Harabasz score. A Silhouette-only ranking would have selected it.

The margin between K-Means and Ward is small: they assign 98.94% of days to the same cluster, and the difference in mean rank is one position on one index. The fair conclusion is that the two methods are equivalent on this data and that K-Means was kept for its simplicity and speed (about 0.12 s per fit); the comparison does not show that K-Means is clearly superior.

6.1.4 What the clusters represent

Table 6.3 describes the two K-Means clusters and Figure 6.3 shows their composition by day of week. Cluster 0 contains every weekday-like day and no weekend days; cluster 1 contains all 564 weekend days and only 17 weekdays. Cluster membership agrees with a simple weekend indicator on 99.14% of days. The mean profiles of the two clusters have the same shape, a small morning rise and an evening peak at 19:30, and differ mainly in level, by about 330 kW.

Table 6.3: Profile of the two K-Means clusters

Characteristic	Cluster 0	Cluster 1
Days	1,396 (70.61%)	581 (29.39%)
Interpretation	Weekday regime	Weekend regime

Characteristic	Cluster 0	Cluster 1
Weekend days in cluster	0	564 (97.07% of the cluster)
Mean demand (kW)	1,839.90	1,510.93
Mean daily peak (kW)	2,723.40	2,275.30
Mean daily minimum (kW)	1,504.53	1,211.61
Peak time of mean profile	19:30	19:30
Isolation Forest anomalies	24	36
Labels: Normal / Peak / Holiday-Low / Abnormal	1,199 / 195 / 0 / 2	80 / 86 / 396 / 19

Source: Evaluation script, from the delivered clustering and label files.

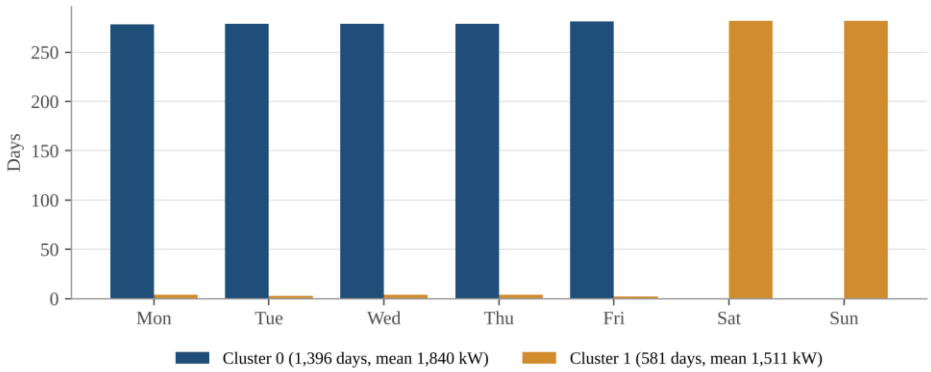


Figure 6.3: Composition of the K-Means clusters by day of week (original in colour)

Source: Author's analysis.

Hypothesis H2 is therefore supported, but in a narrow form: the clustering rediscovers the weekday/weekend split. It does not reveal seasonal or holiday groups, which the proposal had hoped to find, because the series contains a smooth annual cycle in level rather than distinct seasonal shapes, and because it contains no holiday effects to discover.

6.1.5 Behaviour of DBSCAN

The distance from each day to its eighth-nearest neighbour is nearly flat for most days (median 3.72, 95th percentile 4.30) and then rises steeply (97th percentile 19.56, 99th percentile 33.44). The knee lies at the 96.81th percentile, giving eps = 4.4183. At this radius DBSCAN places 1,917 days in one cluster and labels 60 days (3.03%) as noise. Validity indices are undefined for a single cluster, so DBSCAN is reported as not applicable rather than ranked.

The 60 noise days are exactly the 60 days flagged by Isolation Forest. On this data DBSCAN therefore acts as an outlier detector rather than a clustering method. A wider sweep found one setting (min_samples = 4, eps = 3.9007) that produced two clusters with 12.54% noise, but its Silhouette on the clustered days (0.3532) is still below that of K-Means and it discards 248 days, so it does not change the selection.

6.1.6 Comparison of anomaly detectors

Table 6.4 and Figure 6.4 compare the four anomaly detectors; Isolation Forest (rank 1, in bold) was selected. Isolation Forest and LOF each flag 60 days and have the same stability: a mean Jaccard of 0.7769 (standard deviation 0.032) across the 20 subsamples, with a mean symmetric difference of 13.6 days. One-Class SVM flags 75 days and is much less stable (0.2812, 77.9 days). The z-score rule flags no day at threshold 3.0 and is therefore not ranked.

Table 6.4: Comparison of anomaly detectors

Detector	Days flagged	Rate	Mean Jaccard	Std Jaccard	Mean sym. diff. (days)	Overlap with IF	Rank
Isolation Forest	60	3.03%	0.7769	0.0320	13.6	60	**1**
Local Outlier Factor	60	3.03%	0.7769	0.0320	13.6	60	2
One-Class SVM	75	3.79%	0.2812	0.0516	77.9	30	3
Z-score ($\|z\| \geq 3$)	0	0.00%	–	–	–	0	not ranked

Source: models/anomaly_comparison_cache.json, reproduced by the evaluation script.

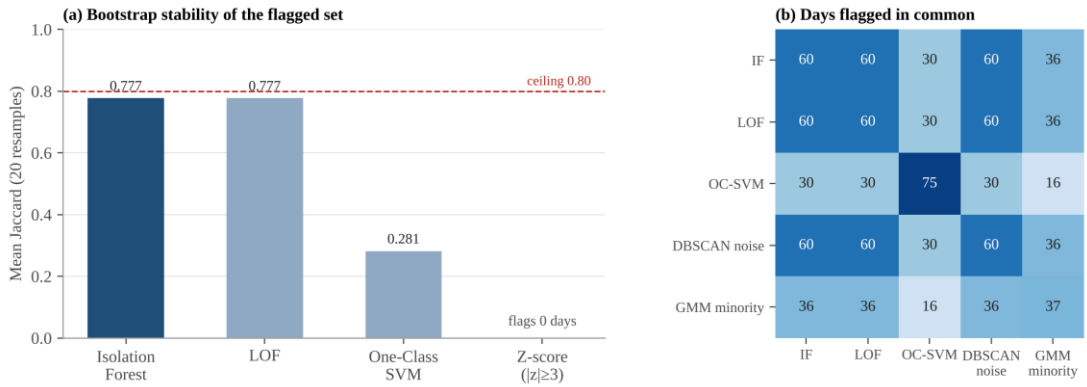


Figure 6.4: Stability of the anomaly detectors and overlap of their flagged days (original in colour)

Source: Author's analysis; values from Table 6.4 and the evaluation script.

Three points qualify these results. First, the selection of Isolation Forest over LOF is a tie: the two flag the same 60 days on the full data and have identical stability in

every subsample, and Isolation Forest is ranked first only because it is listed first. Isolation Forest is a reasonable choice for other reasons, such as its continuous score and speed, but the stability measure does not separate the two.

Second, the stability has a ceiling. Each subsample contains 1,581 days, so a detector tuned to 3% flags about 48 of them, while the full-data set has 60 days. A perfectly stable detector therefore scores 48/60 = 0.80, and 0.7769 indicates near-perfect stability rather than the moderate stability that the comparison page describes.

Third, and most important, methods based on different principles agree. Isolation Forest (random partitioning), LOF (local density) and DBSCAN noise (global density with no contamination parameter) select exactly the same 60 days, and 36 of the Gaussian mixture's 37-day minority component are among them. Because DBSCAN has no preset anomaly rate, this agreement is not forced by the 3% setting. It is the strongest evidence in this study that the 60 days form a real, separate group, and it supports hypothesis H3.

The Spearman correlations between the continuous scores add a further observation: Isolation Forest's ranking of all 1,977 days is highly correlated with the simple z-score (0.9028), yet the z-score threshold never fires, whereas its correlations with LOF (0.2046) and One-Class SVM (0.0767) are low. The detectors agree on the extreme days but rank ordinary days differently.

6.1.7 Characteristics of the anomalous days

The Isolation Forest scores separate cleanly: the lowest score among the 60 flagged days is 0.0449 and the highest among the other days is −0.1154, with no day in between, so the result is not sensitive to the exact contamination value near 3%. Every anomalous day has at least one 15-minute slot below 0.7 times the median profile, and 39 of them also have a slot above 1.5 times the median; no other day has any slot above 1.5 times the median. In the raw data, the daily minimum of anomalous days averages 804 kW against 1,438 kW for other days, and their raw daily maximum averages 3,666 kW against 2,598 kW. The anomalous days are therefore days on which the load jumps between very low and very high values from one reading to the next. Table 6.5 lists the ten highest-scoring days.

Table 6.5: The ten highest-scoring anomalous days

Date	Day	Peak (kW, cleaned)	Mean (kW)	IF score	Final label
2024-07-04	Thu	2,927.77	1,764.16	0.146	Peak Demand Day
2022-06-20	Mon	2,905.67	1,871.34	0.133	Peak Demand Day
2022-01-06	Thu	2,880.97	1,492.24	0.131	Peak Demand Day
2020-09-24	Thu	2,914.87	1,630.58	0.131	Peak Demand Day
2021-03-18	Thu	2,922.50	1,725.04	0.130	Peak Demand Day
2024-02-01	Thu	2,830.60	1,660.49	0.130	Peak Demand Day
2023-05-13	Sat	2,584.85	1,360.00	0.130	Abnormal Demand Day
2020-08-05	Wed	2,923.79	1,772.71	0.129	Peak Demand Day
2023-12-12	Tue	2,877.65	1,669.28	0.129	Peak Demand Day
2024-11-12	Tue	2,736.24	1,626.60	0.128	Abnormal Demand Day

Source: Evaluation script (anomaly catalogue), from the delivered result files.

The anomalous days are spread evenly over the years (10, 10, 14, 9, 12 and 5 from 2020 to 2025), over all days of the week and over all months, and none of the highest-scoring dates coincides with a recognisable Sri Lankan event. Real-world causes cannot therefore be attributed to them, and given the nature of the dataset they are most plausibly artificially injected irregular days. The detector finds them reliably; whether it would find real events of interest to CEB can only be established on real data with an event log.

6.1.8 Peak-demand detection

Table 6.6 shows how many days satisfy each condition of the peak rule. In total 281 days (14.21%) are flagged as peak-demand days. No accuracy figure can be given, because no reference list of peak days exists.

Table 6.6: Conditions of the peak-day rule and the days that satisfy them

Condition	Threshold	Days
Peak demand at or above the 90th percentile	2,884.67 kW	198
Absolute peak z-score above 2.0	2.0	69
Anomalous and peak at or above the 75th percentile	2,819.01 kW	39 (14 not already covered)
Any condition (peak-demand day)		**281 (14.21%)**

Source: data/processed/behavior_labels.csv; evaluation script.

Evaluation revealed a defect in the second condition. The peak z-score is stored as an absolute value, so the condition fires for days whose peak is far *below* the mean as well as far above it. All 69 days that satisfy it have peaks between 1,946.41 and 2,075.50 kW, below the mean daily peak of 2,591.71 kW; all are weekend days; all have mean demand below the low-demand threshold; and none is anomalous. Because no day has a peak more than 1.5 standard deviations above the mean, a correctly signed condition would flag no additional days. The visible symptom is that 39 Saturdays and 39 Sundays are labelled Peak Demand Day, and that the lowest peak among the peak days equals the lowest peak in the whole dataset. Table 6.7 shows the effect of a signed z-score on the label counts.

Table 6.7: Effect of the absolute z-score in the peak rule on the label counts

Label	As implemented (served by the API)	With a signed z-score
Peak Demand Day	281 (14.21%)	212 (10.72%)
Holiday / Low Demand Day	396 (20.03%)	465 (23.52%)
Normal Weekday Demand	1,279	1,279
Abnormal Demand Day	21	21

Source: Evaluation script. The corrected counts are computed, not served by the delivered component.

The correction is a one-line change, but it was identified at the end of the project and was not applied, so that the results reported here match the delivered and demonstrated system. The 69 affected days should be read as low-demand weekend days. The correction is listed as the first item of future work (Section 9.4).

6.1.9 Behaviour labels

Table 6.8 and Figure 6.5 summarise the labels over all 1,977 days.

Table 6.8: Behaviour label distribution and characteristics

Label	Risk	Days	Share	Mean demand (kW)	Mean peak (kW)	Weekend share
Normal Weekday Demand	Normal	1,279	64.69%	1,812.25	2,675.72	6.25%
Peak Demand Day	Medium	281	14.21%	1,780.96	2,691.13	27.76%
Holiday / Low Demand Day	Low	396	20.03%	1,505.66	2,241.86	100%
Abnormal Demand Day	High	21	1.06%	1,513.90	2,742.16	47.62%

Source: data/processed/behavior_labels.csv; evaluation script.

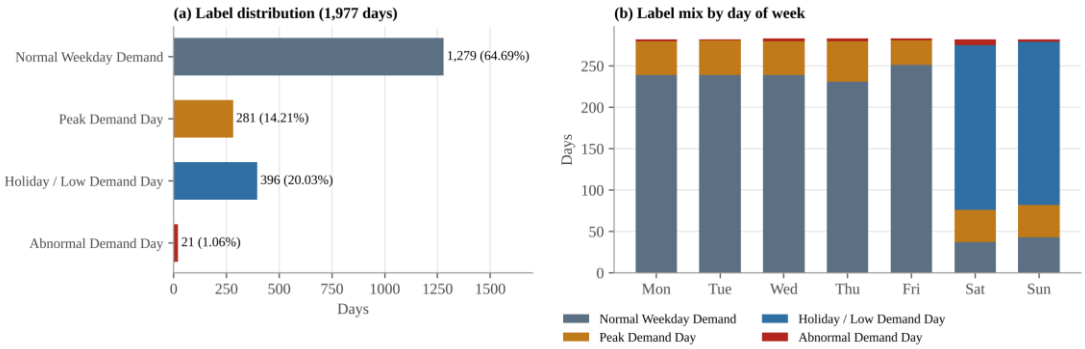


Figure 6.5: Behaviour label distribution and label mix by day of week (original in colour)

Source: Author's analysis; data from the delivered labels file.

Several patterns follow directly from the rules and the data. Every Holiday / Low Demand Day falls on a Saturday or Sunday, and no weekday ever receives this label, so on this dataset the label means an ordinary weekend day. Conversely, 80 weekend days are labelled Normal *Weekday* Demand, because the rule tests demand level and not the calendar. Only 21 of the 60 anomalous days are labelled Abnormal: the other 39 also satisfy the peak rule and are labelled Peak Demand Day by design. The mean peak of the peak-day group (2,691 kW) is barely higher than that of normal days (2,676 kW), because the group is diluted by the 69 low-peak weekend days described in Section 6.1.8; excluding them, the remaining 212 peak days are all high-peak days.

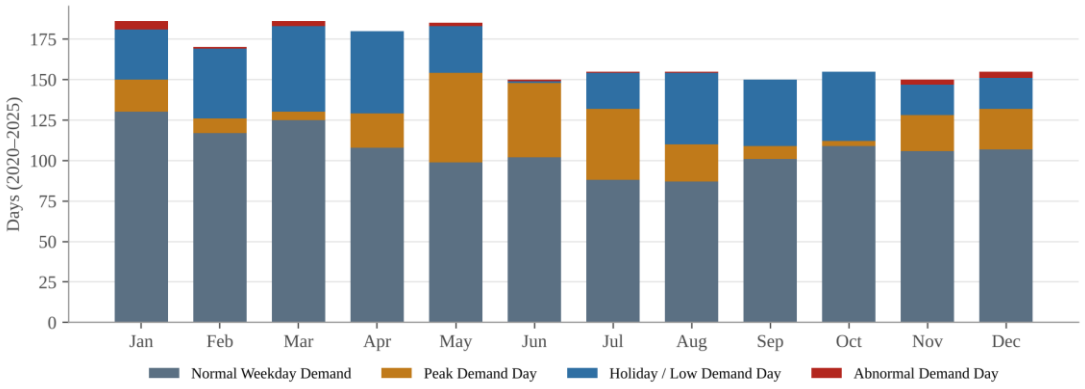


Figure 6.6: Behaviour labels by calendar month, January 2020 to May 2025 (original in colour)

Source: Author's analysis; data from the delivered labels file.

Figure 6.6 shows a clear seasonal pattern in the labels even though the clusters do not: peak days concentrate in May, June and July (55, 46 and 44 days) and are rare in September and October (8 and 3 days), following the annual cycle of demand. The yearly counts are very stable (for example, 50, 55, 51, 51 and 52 peak days from 2020 to 2024), as expected of a series without a trend.

6.1.10 Future-behaviour prediction

Table 6.9 and Figure 6.7 report the back-test of the prediction module on the 364 days from 1 June 2024, using only earlier history. The module reached 78.85% accuracy, against 62.36% for always predicting Normal Weekday Demand, with a macro-averaged F1 of 0.4764. A single prediction takes about 10 ms.

Table 6.9: Back-test of the future-behaviour prediction on a temporal hold-out

Label	Support	Predicted	Precision	Recall	F1
Normal Weekday Demand	227	262	0.8206	0.9471	0.8793
Holiday / Low Demand Day	77	77	0.8312	0.8312	0.8312
Peak Demand Day	57	25	0.3200	0.1404	0.1951
Abnormal Demand Day	3	0	0.0000	0.0000	0.0000
Overall	364		Accuracy 78.85%	Baseline 62.36%	Macro-F1 0.4764

Source: Evaluation script (back-test written for this thesis); the component itself contains no evaluation of this module.

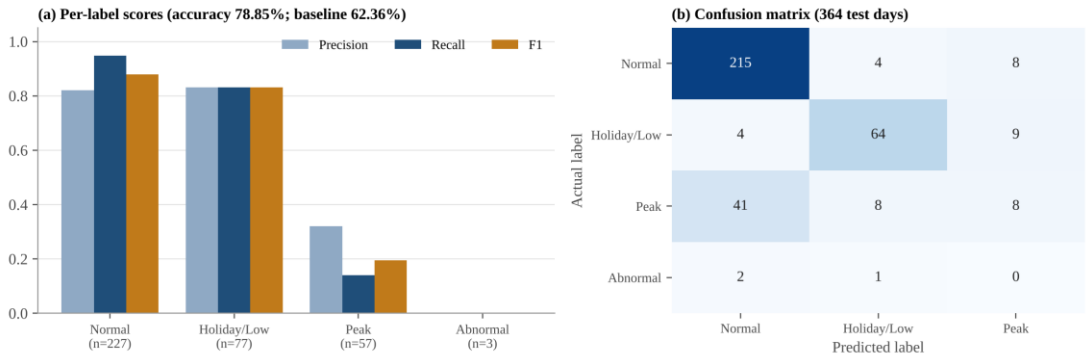


Figure 6.7: Per-label scores and confusion matrix of the prediction back-test (original in colour)

Source: Author's analysis.

The module has learned the weekday/weekend rhythm and little else. It identifies normal weekdays and low-demand weekends well, finds only 8 of the 57 peak days

and never predicts an abnormal day, which is expected: a frequency vote over matched days cannot predict events that are rare and irregular. Its mean reported confidence (84.28%) is higher than its accuracy, so the confidence figure should be read as the share of similar past days, not as a calibrated probability. Hypothesis H4 is supported in that the prediction beats the majority baseline, but the module is a baseline rather than a forecaster of unusual days.

6.2 Product and Technical Results

The delivered product consists of the analysis pipeline, a REST service with 12 endpoints, and a dashboard with five functional areas and two comparison pages, all of which were demonstrated working (Section 5.2). Table 6.10 gives the measured response times of the main endpoints and Figure 6.8 the run time of the pipeline stages.

Table 6.10: Measured API response times

Endpoint	Median (ms)	Range (ms)	Response size
GET /health	6.2	5.6 – 25.4	50 B
GET /api/dashboard-summary	155.5	146.4 – 218.4	524 kB
GET /api/get-day-profile	113.7	109.2 – 143.4	5.0 kB
GET /api/predict-future-behavior	73.0	70.0 – 107.1	0.5 kB
GET /api/download-report	91.7	89.4 – 103.7	357 kB
GET /api/model-comparison (cached / uncached)	8.9 / 1,787	8.2 – 22.1	4.4 kB
GET /api/anomaly-comparison (cached / uncached)	9.5 / 28,147	8.4 – 38.8	4.7 kB
POST /api/run-clustering	8,171	single call	–

Source: API smoke test (in-process client, network time excluded; 20 calls per read endpoint, 5 for the report).

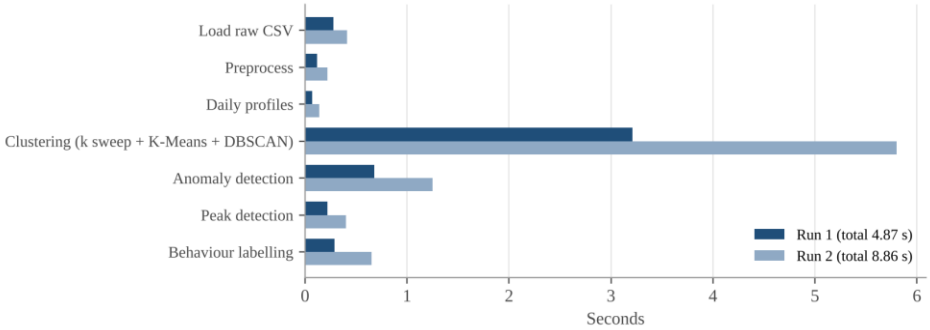


Figure 6.8: Run time of the pipeline stages in two runs (original in colour)

Source: Evaluation script.

The full pipeline runs in 4.9 to 8.9 seconds on a laptop, with clustering taking most of the time because the Silhouette score is computed for eleven values of k; process memory grew from 235 MB to 284 MB during a run. Read endpoints respond in 70 to 160 ms, almost all of which is spent re-reading CSV files on each request. The uncached anomaly comparison takes roughly 15 to 35 seconds because of the 20 refits of each detector, which is why its results are cached; cached responses take under 10 ms. These figures meet the performance requirement for a planning tool used interactively, although the dashboard summary returns all 1,977 days in one 524 kB response, which will not scale to much longer histories without pagination.

6.3 User or Stakeholder Validation

The proposal planned an expert review of the labels with CEB system operators and energy planners. This review was not carried out, and no structured usability evaluation was conducted. [If any feedback was obtained from the supervisor, panel members, teammates or potential users during progress presentations or demonstrations, describe it here, including who gave it, what they were shown and what changed as a result.] Without such validation, the usefulness of the labels to planners remains a design claim rather than a measured result.

6.4 Integration Results

When the system is started with the team launcher, the behaviour intelligence dashboard runs as the shell of the system and opens the Demand Forecasting, Explainable AI and Adaptive Forecasting applications from its sidebar; if a teammate application is not running, the shell shows its owner and start commands instead of

an error. The LIME buttons open the XAI component's anomaly-diagnosis page for the selected date, and that page recognises the request as coming from Behaviour Intelligence.

Three integration gaps were found. The XAI backend reads the earlier dataset file directly from this component's folder, so a LIME explanation opened from an abnormal day describes a different load series for that date. The XAI code expects a source=bi-predict link and a matched_dates field from the prediction service, which this component does not provide. And no teammate component calls this service's API, so behaviour labels are not yet passed to the forecasting services as context, as the proposal's data flow intended. The integration is therefore demonstrated at the user-interface level but not at the data level.

6.5 Summary of Findings

Table 6.11: Summary of findings and strength of evidence

Finding	Evidence	Strength
K-Means with k = 2 gives the best combined clustering quality	Best on all three indices across k; best mean rank across algorithms	Strong for this data; K-Means and Ward effectively equivalent
The clusters are the weekday and weekend regimes	99.14% agreement with a weekend indicator	Strong
Single-index selection would mislead	GMM: highest Silhouette, worst DB and CH; isolates 37 outlier days	Strong
The 60 anomalous days are a real, separate group	IF, LOF and DBSCAN noise identical; clean score gap	Strong internally; correctness against real events not testable
Isolation Forest is preferable to LOF	Identical stability; ranked first by order	Weak (a tie)
Labels describe every day with reasons	1,977 labels, risk levels and reasons	Strong as a capability; usefulness not validated with users
Peak labels include 69 low-demand weekend days	Absolute z-score in the peak rule	Strong (a defect)
Prediction beats the majority baseline	78.85% vs 62.36%; never predicts abnormal days	Moderate
The service is correct and reproducible	24/24 API checks; 0 mismatches on re-run	Strong

Source: Sections 6.1 to 6.4.

7 DISCUSSION

7.1 Interpretation of Findings

RQ1 – Which clustering algorithm groups the profiles best? On combined validity, K-Means with two clusters, with Ward agglomerative clustering practically indistinguishable from it. Hypothesis H1 is supported: the partitioning and hierarchical methods outperform the density-based and mixture models on the combined indices. The more useful lesson concerns how the selection was made. Judged on the Silhouette alone, the Gaussian mixture would have won, although it simply separates 37 extreme days from the rest. Requiring a model to do well on three indices with different definitions of a good partition protected the selection from that failure, and reporting DBSCAN's indices as undefined rather than zero kept the comparison honest.

RQ2 – What do the clusters represent? The weekday and weekend regimes, with 99.14% agreement. This is a correct and meaningful result: the main source of variation between days in this series is whether the day is a working day. It is also a modest one, since a calendar rule would reproduce it. The absence of seasonal or holiday clusters is explained by the data rather than the method. Seasonality in this series changes the level of demand gradually rather than the shape of the day, and a level shift that varies smoothly through the year does not create separate groups; holidays are not represented in the series at all. On real CEB data, where Poya days, festivals, school terms and load shedding change the shape of demand, the same pipeline could find more groups, and the comparison procedure would be needed to judge them.

RQ3 – Which anomaly detector is most stable, and do methods agree? Isolation Forest and LOF are equally and almost perfectly stable, and three methods built on different principles select exactly the same 60 days. Hypothesis H3 is supported. The agreement with DBSCAN noise is particularly informative because DBSCAN has no contamination setting: the set of 60 days is not an artefact of asking every detector for 3%. The result also exposes a limit of the stability criterion: it can separate a stable detector from an unstable one (One-Class SVM), but not two detectors that

find the same thing. Choosing between Isolation Forest and LOF requires other criteria, such as interpretability of the score or behaviour on new days, which this study did not test.

RQ4 – Do the labels give a useful, interpretable description? The labels give every day a readable category, a risk level and a reason that a planner can check, and the prediction module beats a majority baseline. The capability exists and works. Whether the description is *useful* is less certain, for three reasons found in the evaluation: on this dataset the Holiday / Low label is in effect a weekend label; 69 of the 281 peak days are low-demand weekend days because of the absolute z-score; and 39 of the 60 anomalies are reported as peak days rather than as abnormal days. The first is a consequence of the data, the second is a defect, and the third is a design choice that hides anomalies on high-demand days at the highest risk level; it would be better to show both properties. Hypothesis H4 is supported as stated, but the evaluation shows that rule-based labels need to be checked against the data they describe before they are trusted.

A theme runs through all four answers: the most valuable outputs of the evaluation were the checks that went beyond the headline metrics. The clean separation of anomaly scores, the identical flagged sets, the 0.80 stability ceiling, the composition of the clusters by day of week and the composition of the peak class by peak value each changed the interpretation of a number that, on its own, looked either better or worse than it was.

7.2 Comparison with Literature Baselines and Existing Solutions

The numerical results cannot be compared with those of the reviewed studies, which used other data, levels and tasks. The comparison is therefore with what those studies did. Like the Jawa-Bali study [10], this component applies K-Means to whole load profiles and finds groups that correspond to social rhythms of activity; unlike it, it works at day level and combines clustering with anomaly and peak detection. Unlike studies that cluster in order to forecast [6], [7], it uses clusters and anomalies to describe demand directly. Unlike meter-level anomaly frameworks [8], [9], it detects abnormal days of system demand without labels, and it uses agreement between

independent detectors and bootstrap stability [26] to justify the choice of detector instead of relying on the preset anomaly rate. Chicco's review recommends evaluating load-pattern clustering with several indices [13]; the multi-index ranking implemented here follows that advice and shows, with the Gaussian mixture, why it matters.

Compared with existing tools, the component offers something that forecasting software and SCADA displays do not: a day-by-day behavioural reading of demand with reasons, linked to an explanation service. Compared with general-purpose libraries, it packages the methods, the model comparison and the interpretation into one service that a planner can use without writing code.

7.3 Research Contribution

The research contribution is threefold. First, an end-to-end, reproducible pipeline that turns a 15-minute load series into day-level behaviour intelligence by combining clustering, anomaly detection, peak detection and rule-based labelling, which the proposal identified as missing from the literature. Second, a documented procedure for selecting unsupervised models without ground truth: mean rank over three validity indices for clustering, and bootstrap Jaccard stability plus cross-method agreement for anomaly detection, with explicit handling of degenerate and non-firing methods. Third, an evaluation that shows how such a pipeline can mislead and how to detect it: single-index selection, count-based detector comparison, outlier cleaning that removes the events of interest, and absolute-value thresholds.

7.4 Product Contribution

The product contribution is a working behaviour-intelligence service: a FastAPI backend with 12 endpoints, a dashboard that presents labels, risk levels, anomalies, daily profiles and predictions, two comparison pages that explain the model choices to non-specialists, a CSV export, and the integrated shell through which the team's four applications are reached, with deep links from abnormal days to the XAI component. The service is reproducible, passed all 24 API checks and responds interactively.

7.5 Limitations and Threats to Validity

Table 7.1 lists the limitations and threats to validity identified in the evaluation, with their effect and a mitigation.

Table 7.1: Limitations and threats to validity

Limitation	Effect on the results	Mitigation / future work
Synthetic dataset of undocumented provenance	Results show how the component behaves on generated data, not on CEB demand	Obtain and document real utility data; repeat the evaluation
No ground-truth events	No precision, recall or F1 for anomalies or peaks; agreement is evidence of consistency, not correctness	Build a labelled set from holidays, load-shedding records and incident logs
3×IQR cleaning removes spikes	691 high readings on 267 days removed before peak and anomaly analysis; peaks capped at 2,928 kW	Run peak and anomaly detection on uncleaned profiles, or clip instead of delete
Absolute z-score in the peak rule	69 low-demand weekend days labelled Peak Demand Day	Use the signed z-score (Peak 212, Holiday / Low 465)
Clusters equal the weekday/weekend split	Clustering adds little beyond the calendar on this data	Cluster weekdays and weekends separately; add calendar features
Labels ignore clusters and calendar	Holiday label means weekend; 80 weekend days labelled Normal Weekday	Use calendar and cluster information in the rules
Relative thresholds and preset contamination	About 10% peak, up to 25% low and 3% anomalous days by construction	Calibrate thresholds against operational definitions
In-sample evaluation	Models and thresholds fitted and assessed on the same days	Temporal validation; scoring new days with saved models
Isolation Forest vs LOF is a tie	Selection not supported by the stability criterion alone	Compare on further criteria or report both
Prediction is a frequency baseline	Never predicts abnormal days; confidence over-states accuracy	Use forecasts and calendar features from the forecasting components
No user validation	Usefulness to planners not measured	Expert review and usability study with planners
Partial integration and no security	Labels not consumed by other services; no TLS or authentication	Service-to-service API use; gateway with TLS and JWT

Source: Chapters 3 to 6.

The most important limitation is the first. Every quantitative result in this thesis describes the component's behaviour on a series whose structure was simple and whose irregular days were probably injected. The design, the comparison procedures and the software are not tied to this dataset, but the specific findings (two clusters, 60 anomalies, the label shares) are.

7.6 Implications

For planners and utilities, the work shows that day-level behaviour labels with reasons can be produced automatically and presented in a form that does not require machine-learning knowledge, and that such labels should be accompanied by the rule and values behind them so that they can be checked. For researchers, it shows that unsupervised model selection should use several validity indices, should not compare anomaly detectors by counts that are fixed in advance, and benefits from agreement between independent methods; it also shows that preprocessing designed for forecasting, such as aggressive outlier removal, can delete the very events that behaviour analysis is meant to find. For practitioners building similar components, the experience of this project suggests evaluating against the data early, keeping one pipeline as the single source of results, and treating every threshold as a hypothesis to be checked.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The proposition is to turn raw demand data into a day-by-day behavioural reading that planners can act on: what kind of day each day was, which days were abnormal or exceptionally demanding, why each label was given, and what kind of day is likely on a future date. The primary customer persona described in the proposal is a senior load-dispatch engineer at the CEB System Control Centre who prepares generation schedules every day from numerical forecasts but receives no explanation of why demand behaves differently on particular days. For such a user the component offers three outcomes: earlier attention to likely peak days, early warning of abnormal days, and labels that can be verified from the values behind them. Within the project, the component is also the shell through which the forecasting and explanation services are reached, which makes it the natural entry point of the whole product.

8.2 Market and Competitive Position

The primary market identified in the proposal is electricity utilities and grid operators in Sri Lanka, with CEB as the most direct potential adopter. Secondary markets are the Public Utilities Commission of Sri Lanka and ministries responsible for energy planning, for whom behavioural understanding of demand matters as the share of variable renewable generation grows; tertiary markets are large industrial and commercial consumers, smart-grid solution providers in South Asia and energy consultancies. The competitive position claimed is that no product used in Sri Lanka combines day-level clustering, system-level anomaly detection, peak identification and interpretable labelling in one service, and that the component is designed around Sri Lankan demand. That position is only partly earned so far: the component has not been tested on Sri Lankan utility data, and no market study or customer interviews have been carried out [add any market evidence collected]. These claims are therefore hypotheses for a pilot to test.

8.3 Business or Sustainability Model

The proposal envisaged annual software licences for utilities and planning agencies (LKR 250,000 to 450,000 per institution per year), project-based configuration and integration services (LKR 0.5 to 2 million per engagement), cloud-hosted subscriptions for smaller utilities and research institutions (LKR 15,000 to 80,000 per month), and research licensing of the labelling methodology. A sustainable path would begin with a free or discounted pilot with one utility, using its data and event logs to validate the labels, and move to a subscription once the value of the labels to planners has been shown. Running costs are low compared with deep-learning services: the pipeline runs in seconds on a laptop, so hosting, storage and the staff time to maintain the rules and thresholds are the main costs. Table 8.1 gives the project budget planned in the proposal. [Replace or supplement with actual expenditure.]

Table 8.1: Planned project budget (LKR)

Item	Quantity	Unit cost	Total
High-performance laptop	1	185,000	185,000
External SSD	1	18,500	18,500
Cloud GPU instance	8 months	6,800	54,400
Backend hosting server	10 months	3,200	32,000
Domain and SSL certificate	1	7,500	7,500
Cloud data backup and storage	10 months	2,200	22,000
Data collection and CEB coordination	4 visits	4,500	18,000
Visualisation and dashboard tools	1	8,500	8,500
Documentation and report preparation	1	12,500	12,500
Contingency reserve	1	15,000	15,000
Total			**373,400**

Source: Project proposal, Table 4.

8.4 Scalability, Intellectual Property and Compliance

Scalability. The algorithms scale to many years of daily profiles, but the current service does not: it runs as a single process, re-reads CSV files on every request, and returns all days in one response. Commercial use would require a database, pagination, in-memory caching and the ability to score new days with the saved

models as data arrive. Serving several utilities or regions would mean separate model sets per client. **Intellectual property.** The algorithms are public and the implementation uses open-source libraries, so protectable value lies in the source code, the labelling taxonomy and rules, the model-comparison procedures and their calibration to a utility's data, which are protected by copyright rather than patents. Ownership under SLIIT policy and any data-provider agreements should be confirmed [confirm]. **Compliance.** The component uses aggregated system demand with no personal data, but any deployment must respect the Personal Data Protection Act, No. 9 of 2022 [32], contractual limits on utility data, and the sector regulator's requirements, and must add the authentication and encryption planned in the proposal. Labels should always be presented with their reasons and limitations so that they are not over-relied upon in operational decisions.

8.5 Contribution to the Sustainable Development Goals

- **SDG 7 (Affordable and Clean Energy).** Understanding which days are normal, peak or abnormal helps utilities plan generation and reserves, supporting reliable supply and the integration of variable renewable energy.
- **SDG 9 (Industry, Innovation and Infrastructure).** The component applies interpretable machine learning to critical energy infrastructure in a form that can be adopted by local utilities.
- **SDG 13 (Climate Action).** Better preparation for peak and abnormal days can reduce reliance on expensive and polluting peaking plants; the size of this effect has not been measured in this study.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the Behaviour Intelligence component of the AI-Based Electricity Demand Intelligence System for Sri Lanka. From a 15-minute load series of 189,793 readings, the component builds 1,977 daily 96-point profiles and gives every day a cluster, an anomaly score, a peak flag and one of four behaviour labels with a risk level and a written reason, and it serves these through a FastAPI service and a React dashboard that also integrates the team's other applications.

Four clustering algorithms were compared by their mean rank on three validity indices. K-Means with two clusters was selected (Silhouette 0.4729, Davies–Bouldin 0.8373, Calinski–Harabasz 1,367.33), with Ward agglomerative clustering practically equivalent, and the comparison showed that a Silhouette-only choice would have selected a Gaussian mixture that merely isolates outliers. The two clusters are the weekday and weekend regimes. Four anomaly detectors were compared by bootstrap stability; Isolation Forest was selected in a tie with LOF, with near-ceiling stability (0.7769 against 0.80), and three independent methods agreed on exactly the same 60 abnormal days. The labelling produced 1,279 normal, 281 peak, 396 holiday/low and 21 abnormal days, and the prediction module reached 78.85% accuracy on a temporal hold-out against a 62.36% baseline. The service passed all 24 API checks and its results were reproduced exactly.

The evaluation also established the limits of these results. The dataset is synthetic and of undocumented provenance; there are no ground-truth events, so detection accuracy is unmeasured; outlier cleaning removes demand spikes before peak analysis; the peak rule's absolute z-score labels 69 low-demand weekend days as peak days; and the labels have not been validated with planners. Within these limits, the work provides a reproducible pipeline, a justified way to choose unsupervised models without labels, and a usable service that turns demand data into readable day-level behaviour intelligence.

9.2 Achievement of Objectives

Table 9.1: Achievement of the objectives

Objective	Status	Evidence
1. Preprocess the raw data	Achieved, with a limitation	Section 3.2.4; cleaning removes 691 high readings
2. Build daily profiles	Achieved	1,977 profiles of 96 values with peak, mean and standard deviation
3. Cluster and select a model	Achieved	Table 6.1, Table 6.2; K-Means k = 2 by mean rank
4. Detect anomalies and select a detector	Achieved	Table 6.4; Isolation Forest by stability; cross-method agreement
5. Labels, API and dashboard	Achieved, with a defect in the peak rule	Table 6.8, Table 4.5, Section 5.2; Table 6.7
6. Evaluate the component	Partly achieved	Internal validity, stability, back-test and software tests done; precision/recall and expert review not possible or not done

9.3 Individual Research and Product Contributions

The research contribution is a day-level behaviour-intelligence pipeline with documented, ground-truth-free procedures for selecting the clustering model and the anomaly detector, together with an evaluation that identifies how such pipelines can mislead. The product contribution is the behaviour-intelligence service and dashboard, including the model-comparison pages, the prediction panel, the CSV export and the integrated shell of the team's system.

9.4 Future Directions

- Correct the peak rule to use the signed z-score, regenerate all outputs from the API pipeline, and retire or realign the older training script so that there is one source of results.
- Run peak and anomaly detection on profiles before spike removal, or clip extreme values instead of deleting them.
- Obtain real CEB or LECO data with documented provenance, and build a labelled set of events (public and Poya holidays, load-shedding periods, outages) to measure precision, recall and F1.

- Add calendar features so that Weekday and Holiday labels follow the calendar, use the cluster assignment in the rules, and cluster weekdays and weekends separately to look for structure beyond the level shift.
- Show anomalies on high-demand days as both peak and abnormal rather than hiding the anomaly.
- Score new days with the saved models through an endpoint, so that the service can label incoming data in near real time.
- Feed behaviour labels to the forecasting components as context and align the interface with the XAI component (shared dataset, matched dates).
- Add automated tests, logging, input validation, proper HTTP status codes, pagination, a database, and TLS with token-based authentication.
- Carry out an expert review and a usability study with planners and operators.

REFERENCES

[1] International Energy Agency (IEA), "Electricity 2024: Analysis and forecast to 2026," IEA, Paris, France, 2024.

[2] D. N. L. S. Uduwage, R. K. Shiwakoti, and P. Limcharoen, "Short-term electricity demand forecasting in Sri Lanka using statistical and deep learning models," in Proc. 13th World Construction Symposium, Sri Lanka, Aug. 2025.

[3] G. V. B. De Silva and L. A. Samaliarachchi, "Peak electricity demand prediction model for Sri Lanka power system," ENGINEER, Journal of the Institution of Engineers, Sri Lanka, vol. 46, no. 4, pp. 53–60, 2013.

[4] P. O. Ugbehe, O. E. Diemuodeke, and D. O. Aikhuele, "Electricity demand forecasting methodologies and applications: A review," Sustainable Energy Research, vol. 12, art. 19, 2025, doi: 10.1186/s40807-025-00149-z.

[5] Z. Omar and L. Liyanage, "A meta-analysis of electricity demand forecasting using modelling methods, factors, and horizon: A review study of Australia, Pakistan, and Sri Lanka," Sustainable Energy Research, 2025. [Online]. Available: https://ssrn.com/abstract=4791831

[6] X. Jin, X. Wang, and Y. Liu, "Short-term load forecasting based on the analysis of user electricity behavior," Algorithms, vol. 9, no. 4, art. 80, 2016.

[7] Y. Kim, J. Lee, and K. Choi, "Time-series clustering and forecasting household electricity demand using smart meter data," Energy Reports, vol. 9, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352484723002731

[8] S. Maleki, S. Maleki, and N. R. Jennings, "Anomaly detection with machine learning algorithms and big data in electricity consumption," Sustainability, vol. 13, no. 19, art. 10963, 2021.

[9] L. Mascali, D. S. Schiera, S. Eiraudo, L. Barbierato, R. Giannantonio, E. Patti, L. Bottaccioli, and A. Lanzini, "A machine learning-based anomaly detection framework for building electricity consumption data," Sustainable Energy, Grids and Networks, vol. 36, art. 101194, 2023.

[10] "Electricity load classification using K-means clustering algorithm," in Proc. 2020 Int. Conf. Technology and Policy in Energy and Electric Power (ICT-PEP), 2020, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9249773 [author names to be added].

[11] "Unsupervised grouping of industrial electricity demand profiles: Synthetic profiles for demand-side management applications," Energy, 2021. [Online]. Available:

https://www.sciencedirect.com/science/article/pii/S0360544220320697 [author names to be confirmed].

[12] "DBSCAN-based electricity consumption anomaly detection method integrated with VAE," Engineering Reports, 2025, doi: 10.1002/eng2.70183 [author names to be confirmed].

[13] G. Chicco, "Overview and performance assessment of the clustering methods for electrical load pattern grouping," Energy, vol. 42, no. 1, pp. 68–80, 2012.

[14] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symp. Mathematical Statistics and Probability, vol. 1, 1967, pp. 281–297.

[15] A. K. Jain, "Data clustering: 50 years beyond K-means," Pattern Recognition Letters, vol. 31, no. 8, pp. 651–666, 2010.

[16] J. H. Ward, "Hierarchical grouping to optimize an objective function," Journal of the American Statistical Association, vol. 58, no. 301, pp. 236–244, 1963.

[17] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD-96), 1996, pp. 226–231.

[18] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.

[19] D. L. Davies and D. W. Bouldin, "A cluster separation measure," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[20] T. Caliński and J. Harabasz, "A dendrite method for cluster analysis," Communications in Statistics, vol. 3, no. 1, pp. 1–27, 1974.

[21] V. Satopää, J. Albrecht, D. Irwin, and B. Raghavan, "Finding a 'kneedle' in a haystack: Detecting knee points in system behavior," in Proc. 31st Int. Conf. Distributed Computing Systems Workshops, 2011, pp. 166–171.

[22] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, art. 15, 2009.

[23] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.

[24] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in Proc. ACM SIGMOD Int. Conf. Management of Data, 2000, pp. 93–104.

[25] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," Neural Computation, vol. 13, no. 7, pp. 1443–1471, 2001.

[26] C. Hennig, "Cluster-wise assessment of cluster stability," Computational Statistics & Data Analysis, vol. 52, no. 1, pp. 258–271, 2007.

[27] M. Gonzalez-Torres, L. Perez-Lombard, and J. F. Coronel, "Analysis and forecasting of monthly electricity demand time series using pattern-based statistical methods," Energies, vol. 16, no. 2, art. 827, 2023.

[28] R. K. Shiwakoti, P. Limcharoen, and D. N. L. S. Uduwage, "Electricity user behavior analysis and marketing strategy based on internet of things and big data," Energy Informatics, 2024, doi: 10.1186/s42162-024-00397-1.

[29] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[30] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004.

[31] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007.

[32] Parliament of the Democratic Socialist Republic of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," Colombo, Sri Lanka, 2022.

[33] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.

Note: entries [author names to be added/confirmed] were carried over from the project proposal with incomplete details. Each entry must be checked against the original publication before submission.

GLOSSARY

Behaviour label – One of four categories (Normal Weekday Demand, Peak Demand Day, Holiday / Low Demand Day, Abnormal Demand Day) assigned to each day by ordered rules.

Bootstrap stability – Agreement of a method's output across repeated subsamples of the data; here the mean Jaccard overlap between each subsample's flagged days and the full-data flagged days.

Contamination – The expected fraction of anomalies given to Isolation Forest and LOF in advance (0.03 here); it fixes how many points they flag.

Daily load profile – The 96 load values of one day at 15-minute intervals.

Degenerate clustering – A result with fewer than two clusters, for which validity indices are undefined.

Elbow method – Choosing k where the within-cluster sum of squares stops falling sharply.

Jaccard coefficient – The size of the intersection of two sets divided by the size of their union.

k-distance knee – The bend in the sorted distances of every point to its k-th nearest neighbour, used to choose DBSCAN's radius.

Mean rank – The average of a model's ranks on the Silhouette, Davies–Bouldin and Calinski–Harabasz indices; the clustering selection criterion.

Poya day – A monthly Buddhist full-moon public holiday in Sri Lanka, which affects electricity demand.

Risk level – A fixed attention level attached to each label: Normal, Low, Medium or High.

Symmetric difference – The number of items in exactly one of two sets.

Validity index – A measure of clustering quality computed without labels.

APPENDICES

Appendix A Extended Results and Evidence Record

This appendix compares the proposal plan with the implementation, lists the project artefacts that hold the evidence reported in this thesis, and gives example responses of the service.

A.1 Proposal plan compared with the implementation

Table A.1: Proposal plan compared with the implemented component

Proposal item	Implemented	Note
CEB 15-minute data with holiday and seasonal flags	Synthetic 15-minute series, load only	Source to be confirmed
K-Means and DBSCAN	Both, plus Ward and Gaussian mixture for comparison	DBSCAN degenerate
k by Silhouette and elbow	Silhouette; elbow computed in evaluation	Elbow suggests 3, indices 2
Isolation Forest and z-score	Both; final flag from Isolation Forest	z-score cannot fire at 3.0 on this data
Labels from clusters, scores, peaks and calendar	Labels from anomaly flag, peak flag and demand level	Clusters stored but unused; no calendar
Precision, recall, F1 against real events	Not possible	No event labels
Expert review with CEB operators	Not done	Future work
Matplotlib/Seaborn plots and calendar maps	Interactive React dashboard	No calendar map
Database storage	CSV, JSON and pickle files	Future work
TLS and JWT security	Not implemented	Future work
–	Model-comparison and anomaly-comparison pages, prediction, CSV export, integrated shell	Added beyond the proposal

Source: Project proposal and component code.

A.2 Evidence sources in the project repository

Table A.2: Project artefacts that hold the reported evidence

Evidence	Artefact
Dataset	behaviour-intelligence/data/raw/synthetic_realistic_load.csv
Daily profiles and labels	data/processed/daily_profiles.csv, behavior_labels.csv

Evidence	Artefact
Clustering metrics	models/clustering_metrics.json
Clustering comparison	models/model_comparison_cache.json
Anomaly comparison	models/anomaly_comparison_cache.json
Saved models	models/profile_scaler.pkl, kmeans_model.pkl, dbscan_model.pkl, isolation_forest_model.pkl
Earlier-dataset diagnosis	reports/model_comparison.md
Evaluation scripts	thesis_evidence/scripts/eval_all.py, api_smoke_test.py
Evaluation tables and figures	thesis_evidence/tables (T01–T23), thesis_evidence/figures (F01–F13)
Evidence summary	THESIS_EVIDENCE_IT22545176.md

A.3 Example service responses

A prediction request for a future date:

```
GET /api/predict-future-behavior?date=2026-10-12
{"success": true,
 "message": "Prediction for 2026-10-12 generated.",
 "data": {
   "predicted_label": "Normal Weekday Demand",
   "confidence_percent": 100.0,
   "risk_level": "Normal",
   "matched_days_count": 22,
   "match_strategy": "month + day-of-week",
   "probability_distribution": {
     "Abnormal Demand Day": 0.0, "Peak Demand Day": 0.0,
     "Holiday / Low Demand Day": 0.0, "Normal Weekday Demand": 100.0},
   "explanation": "Based on 22 historical days matched via 'month +
     day-of-week' (month=10, dayofweek=0), 'Normal Weekday Demand'
     appeared 22 times (100.0% of matched days)."}}
```

A day-profile request for an abnormal day (profile shortened):

```
GET /api/get-day-profile?date=2023-05-13
{"success": true,
 "message": "Profile for 2023-05-13 loaded.",
 "data": {
   "date": "2023-05-13",
   "profile_points": [
     {"time_label": "00:00", "slot": 0, "value": 1995.42},
     {"time_label": "00:15", "slot": 1, "value": 2094.69}, ...],
   "peak_demand": 2584.85, "mean_demand": 1360.0,
   "behavior_label": "Abnormal Demand Day",
   "behavior_risk_level": "High",
   "behavior_reason": "Flagged as anomaly (iso_score=0.130,
     peak_zscore=0.03) but not a peak day"}}
```

The CSV report filtered to abnormal days returns 21 data rows with the columns date, behavior_label, behavior_risk_level, peak_demand, mean_demand, is_peak_demand_day, final_is_anomaly, isolation_anomaly_score, kmeans_cluster and behavior_reason; for example, 29 January 2020 is reported as an Abnormal Demand Day with high risk, a peak of 2,799.81 kW, an anomaly score of 0.1155 and the reason *Flagged as anomaly (iso_score=0.116, peak_zscore=0.81) but not a peak day*.

Appendix B Academic Integrity and AI Use Disclosure

In line with the module policy, this appendix discloses the use of generative AI. The author remains responsible for the accuracy, originality, citations, data and conclusions of the thesis.

Table B.1: Disclosure of generative AI use

Item	Disclosure
Tool	Claude (Anthropic), a generative AI assistant
Purpose	Gathering evidence from the author's code (running the author's pipeline in a scratch copy, measuring results, writing evaluation scripts), drafting and structuring thesis text, tables, charts and diagrams
Material provided to the tool	The component source code and result files, the project proposal, screenshots of the running component, and a teammate's thesis as a formatting reference
Verification performed	[The author to describe how every statement, number, citation and figure was checked against the code, result files and sources]
Sections affected	[State the sections drafted or edited with AI assistance]
Content not generated by AI	The component's design and code, its data and its saved results, which were produced by the author